

GOBERNACIÓN AUTÓNOMA DEPARTAMENTAL DE ORURO

AGENDA.GOBER

Documentación Consolidada de Arquitectura de Software
Informe Técnico y Evaluación Crítica de Dirección



Alcance: Backend (NestJS + PostgreSQL/RLS), Frontend (Next.js), Infraestructura y Seguridad

Fecha de corte de la revisión: 12 de septiembre de 2026

Clasificación: Uso interno — Documentación técnica

Método: Auditoría de código fuente, 26 migraciones SQL, 25 diagramas UML/C4 validados y renderizados,
contraste contra reporte de pentest existente

Índice

Resumen ejecutivo

0. Alcance y metodología de esta revisión

1. Propósito del sistema, roles y modelo de autorización

- 1.1 Problema de negocio que resuelve
- 1.2 Los 7 roles y qué puede hacer cada uno
- 1.3 Candado real vs. botón oculto
- 1.4 Estado de implementación por funcionalidad

2. Arquitectura del sistema y catálogo de módulos

- 2.1 Vista general y decisiones de diseño
- 2.2 Ciclo de vida de una petición HTTP
- 2.3 Tiempo real (WebSocket)
- 2.4 Tareas programadas (sweeps)
- 2.5 Archivos, caché y validación
- 2.6 Catálogo de los 18 módulos backend
- 2.7 Frontend
- 2.8 Discrepancias documentación vs. código

3. Modelo de datos

- 3.1 Identidad y organización
- 3.2 Contenido institucional
- 3.3 Despacho y trazabilidad
- 3.4 Patrón de automatización repetido

4. Recorridos de extremo a extremo

5. Hallazgos técnicos y plan de mejoras (10 puntos con evidencia)

6. Puntos críticos — Evaluación de Dirección de Arquitectura

- 6.1 Fortalezas reales (lo que no hay que romper)
- 6.2 Riesgos críticos no cubiertos por el informe técnico
- 6.3 Checklist de "go/no-go" antes de producción real
- 6.4 Hoja de ruta recomendada por fases
- 6.5 Veredicto de dirección

7. Anexo — Catálogo completo de diagramas (25)

Resumen ejecutivo

AGENDA.GOBER es el sistema interno de gestión y coordinación institucional de la Gobernación Autónoma Departamental de Oruro: agenda, publicaciones, tareas, proyectos, la cadena de mando del Despacho del Gobernador y la coordinación de comunicación institucional (UNICOM), todo bajo un único principio de diseño que domina el resto de la arquitectura: **la autorización real vive en la base de datos (Row Level Security de PostgreSQL), no en el código de la aplicación**. Esta decisión, poco frecuente en sistemas de este tamaño, ya fue puesta a prueba con un pentest real y quedó documentada.

Este documento consolida en un solo lugar la carpeta completa de arquitectura (`docs/arquitectura/`): seis documentos narrativos, 25 diagramas C4/UML validados con el compilador real de PlantUML, y un listado de 10 hallazgos técnicos con evidencia y plan de remediación — y le agrega, en la [Sección 6](#), una capa adicional que no estaba en el material original: una lectura de **dirección de arquitectura**, con foco en continuidad de negocio, riesgo organizacional y secuenciación ejecutiva, no solo corrección técnica línea por línea.

Lo que está objetivamente bien resuelto

- Aislamiento multi-secretaría reforzado por la base de datos, no por el frontend (verificado con `FORCE ROW LEVEL SECURITY` y un pentest real).
- Automatización de notificaciones y SLAs sin intervención humana (triggers + barridos programados).
- Tiempo real que reutiliza la misma política de autorización que el HTTP — no hay una segunda copia de la regla que se pueda desincronizar.
- Documentación de arquitectura completa, verificada línea por línea contra el código — una rareza positiva.

Lo que un director de arquitectura no dejaría pasar sin plan

- No hay evidencia de alta disponibilidad de PostgreSQL, ni de un simulacro de restauración de backup.
- No existe ambiente de staging ni pipeline de despliegue continuo — el salto a producción real es, hoy, un salto sin red.
- Alta concentración de conocimiento (RLS + PL/pgSQL + triggers) en muy poca gente — riesgo de "bus factor".
- Sin confirmación explícita de protección CSRF ni de rotación de secretos — no cubierto por el pentest documentado.

El sistema **no está a medio construir**: todos los módulos catalogados como "completos" tienen controller, service, política RLS y (salvo Comunicación) su canal de tiempo real. La deuda técnica es conocida y ya está reconocida por quien construyó el sistema en los propios comentarios de las migraciones — lo cual, en sí mismo, es una señal de madurez de ingeniería poco común. El riesgo real no es de calidad de código: es de **preparación operativa para producción** (continuidad, respaldo, gobernanza de despliegue) y de **concentración de conocimiento crítico**.

0. Alcance y metodología de esta revisión

Este documento reorganiza y consolida en un PDF único la carpeta `docs/arquitectura/` del repositorio, generada a partir de una revisión completa de código (no de una descripción de intenciones). Cada afirmación del material original está clasificada con una de estas cuatro marcas, que se conservan aquí porque son la base de la confianza que se le puede dar a cada frase:

Marca	Qué significa
VERIFICADO EN CÓDIGO	Se leyó el archivo fuente (o se corrió una consulta/comando) y la afirmación describe exactamente lo que ahí está.
DOCUMENTADO SIN VERIFICAR	Sale de un README o comentario del propio repositorio, sin contraste línea por línea contra el código.
INFERENCIA	No está escrito en ningún lado; se dedujo de evidencia indirecta (nombres de tablas, patrones repetidos). Siempre con su evidencia explicada.
PROPUESTA	Es una recomendación, no algo que exista hoy. Nunca se mezcla con la descripción del estado actual.

Cobertura de la revisión

Área	Cobertura
Migraciones SQL (<code>db/migrations/001-026</code>)	Completa — las 26 leídas de punta a punta, incluidas políticas RLS, triggers y funciones <code>SECURITY DEFINER</code> .
Backend — 18 módulos de negocio	Completa — controller, service y DTOs de cada uno.
Backend — transversal (contexto, auth, common, observabilidad)	Completa.
Frontend — enrutamiento y núcleo de sesión/tiempo real	Completa.
Frontend — páginas de módulo	Parcial y dirigida — se leyeron completas Auditoría, Reuniones, Agenda, Tareas, Comunicación y el dashboard de Publicaciones; el resto se confirmó por su contrato con la API, no línea por línea.
CI/CD y despliegue (Docker, workflows)	Completa.
Seguridad	Completa respecto al reporte de pentest existente (no se repitió el pentest).
Pruebas automatizadas	Inventariadas, no re-ejecutadas.

Fuera de alcance, a propósito

- No se leyeron las ~26 páginas de frontend línea por línea (se priorizó el backend, donde vive la autorización real).
- No se ejecutó la suite de tests como parte de esta auditoría documental.
- No se inspeccionó el contenido de `backups/` (artefactos binarios, no código).
- No se re-ejecutó el pentest — se usó como evidencia de lo que ya fue puesto a prueba.

Validación real de los diagramas. Los 25 diagramas PlantUML no se validaron solo a simple vista: se compilaron con `plantuml-1.2024.8.jar` (Graphviz embebido). En la primera pasada, 5 de 25 fallaban por un problema de codificación (el JVM local interpreta por defecto Windows-1252, no UTF-8, y corrompe el guion largo “—” dentro de etiquetas entre comillas); se resolvió forzando `-charset UTF-8`. Se encontraron y corrigieron además 2 errores reales de sintaxis (corchetes anidados en un componente, y una cadena de flechas `A→B→C` no soportada en diagramas de componentes). Resultado final: **los 25 compilan sin error** y cada uno tiene su `.svg` ya renderizado junto al `.puml` editable.

Esta es una fotografía tomada el 2026-09-12. Al momento de la revisión existían cambios sin commitear en el árbol de trabajo — esta documentación describe el código tal como está en disco, no una versión etiquetada específica.

1. Propósito del sistema, roles y modelo de autorización

1.1 Problema de negocio que resuelve INFERENCIA

No existe en el repositorio un documento formal de "visión de producto"; esto se deduce del organigrama modelado en la migración de secretarías reales y de la secuencia de construcción de los módulos. AGENDA.GOVER resuelve tres problemas concretos de una Gobernación departamental:

1. **Aislamiento de información entre secretarías** — cada secretaría gestiona su propia agenda, publicaciones, tareas y proyectos sin que otra secretaría pueda leerlos, salvo el Gobernador y su Gabinete, que ven todo. Resuelto con Row Level Security en la base de datos, no solo con lógica de aplicación.
2. **Cadena de mando y seguimiento (Despacho)** — el Gobernador emite instrucciones que Gabinete desglosa en trabajo concreto de las secretarías, con validación de cierre y trazabilidad de quién hizo qué.
3. **Comunicación institucional coordinada** — la Unidad de Comunicación (UNICOM) coordina con Gabinete qué actividades de las secretarías requieren cobertura de prensa/redes.

Sobre esa base creció con módulos operativos de uso diario (Publicaciones, Agenda, Reuniones, Tareas, Proyectos) y módulos de agregación para la conducción (Gabinete, Indicadores, Auditoría, "Mi bandeja").

Objetivo general INFERENCIA CON EVIDENCIA DIRECTA

Dar a cada persona de la Gobernación una sola pantalla de trabajo que muestre exactamente lo que le corresponde según su secretaría y su rol, sin depender de que la aplicación "se acuerde" de ocultar algo: la restricción vive en la base de datos. VERIFICADO el usuario de aplicación (`app_user`) nunca es superusuario ni tiene `BYPASSRLS` (`CREATE ROLE ... NOSUPERUSER NOCREATEDB NOCREATEROLE NOBYPASSRLS`), y cada tabla de negocio tiene `ENABLE ROW LEVEL SECURITY + FORCE ROW LEVEL SECURITY` — este último es la clave técnica: sin `FORCE` , el dueño de la tabla igual vería todo; con `FORCE` , ni siquiera el dueño escapa a las políticas al conectarse como `app_user` .

1.2 Los 7 roles del sistema

Los primeros tres roles están atados a una secretaría (`ambito_secretaria = true`); los otros cuatro son transversales — ven y actúan a través de todas las secretarías, cada uno con su propio recorte.

Rol	Ámbito	Rango	Quién es INF.
<code>operador</code>	Su secretaría	1 (más bajo)	Personal operativo — carga contenido, ejecuta tareas asignadas.
<code>director</code>	Su secretaría	2	Mando medio — aprueba/edita dentro de su secretaría.
<code>secretario</code>	Su secretaría	3 (más alto de los 3)	Máxima autoridad de la secretaría — único que publica comunicados.
<code>unicom</code>	Transversal (comunicación)	—	Unidad de Comunicación — agenda institucional y gestión de cobertura.
<code>jefe_gabinete</code>	Transversal	—	Coordina el Gabinete — organiza y valida el trabajo del Despacho.

gobernador	Transversal	—	Emite instrucciones, ve todo, reabre instrucciones cerradas.
admin	Transversal (solo sistema)	—	Súper administrador de cuentas/roles — a propósito, no ve contenido estratégico.

El "rango" (`rol_rango()`) solo aplica a los 3 roles de secretaría: `operador(1) < director(2) < secretario(3)` . Un nivel de confidencialidad (`publica / interna` → rango 1, `reservada` → 2, `confidencial` → 3) exige que el rango del usuario sea igual o mayor para ver/crear/editar esa fila. Los 4 roles transversales no tienen techo: cada política RLS los deja pasar sin comparar rangos.

Por qué `admin` está separado de "transversal con acceso total". `admin` puede gestionar usuarios y secretarías, pero no aparece en las políticas RLS de contenido salvo como alias de "acceso total" junto a `gobernador / jefe_gabinete` en varias de ellas. La separación es organizacional: solo `admin` — ni `gobernador` ni `jefe_gabinete` — pasa el `RolesGuard('admin')` de `/admin/usuarios` .

Matriz de capacidades por módulo

`Sí` = puede hacerlo. `Sí¹` = solo dentro de su propia secretaría y sin superar el rango de confidencialidad permitido. `Ver` = solo lectura. `—` = no accesible (RLS deniega por defecto — la fila ni figura).

Módulo	operador	director	secretario	unicom	jefe_gab.	gobernador	admin
Publicaciones — crear/pedir revisión	Sí¹	Sí¹	Sí¹	—	Sí	Sí	Sí
Publicaciones — aprobar	—	Sí¹	Sí¹	—	Sí	Sí	Sí
Publicaciones — publicar	—	—	Sí¹	—	Sí	Sí	Sí
Agenda — ver/crear	Sí¹	Sí¹	Sí¹	Ver	Sí	Sí	Sí
Agenda — editar/borrar	—	Sí¹	Sí¹	—	Sí	Sí	Sí
Reuniones — editar acta/compromisos	—	Sí¹	Sí¹	—	Sí	Sí	Sí
Reuniones — cerrar mi compromiso	Sí (mío)	Sí (mío)	Sí (mío)	—	Sí	Sí	Sí
Tareas — ver/mover mi asignada	Sí (mía)	Sí (mía)	Sí (mía)	—	Sí	Sí	Sí
Tareas — crear/editar completa	Sí¹	Sí¹	Sí¹	—	Sí	Sí	Sí
Proyectos — ver / editar	Sí¹ / —	Sí¹	Sí¹	—	Sí	Sí	Sí
Comunicación — pedir / planificar cobertura	—	Sí¹	Sí¹	Sí	Ver	Ver	—
Comunicación — visto de Gabinete	—	—	—	—	Sí	Sí	—
Despacho — emitir instrucción	—	—	—	—	—	Sí	—
Despacho — organizar/validar ítems	—	—	—	—	Sí	Sí (+reabrir)	—
Despacho — mi ítem	Sí (mío)	Sí (mío)	Sí (mío)	—	—	—	—
Gabinete / Indicadores / Auditoría	Sí¹/Sí¹/—	Sí¹/Sí¹/—	Sí¹/Sí¹/—	—	Sí/Sí/Sí	Sí/Sí/Sí	Sí*/Sí/Sí
Admin — usuarios/secretarías	—	—	—	—	—	—	Sí

1.3 La distinción más importante del sistema: candado real vs. botón oculto

Candado real — una política RLS de Postgres, un trigger que revienta la transacción con `RAISE`

Botón oculto — una condición en el JSX/TSX del frontend (`rangoDeRol(...)`) que solo evita

EXCEPTION , o el RolesGuard de NestJS (solo en /admin/*). Se cumplen aunque la petición no venga del frontend: contra la API con curl , o con una consulta SQL directa usando las credenciales de app_user .

mostrar un control que el backend igual rechazaría. Cita textual del propio código: "No es la autorización real — eso lo decide siempre Postgres (RLS + trigger). Esto únicamente evita mostrar botones que el backend va a rechazar de todas formas."

Únicas tres excepciones donde la barrera real no es una política RLS: usuarios / usuario_roles (se consultan antes de que exista contexto de sesión — problema de huevo y gallina de la autenticación), sesiones (mismo motivo) y /admin/usuarios + /admin/secretarias (autorización por RolesGuard de NestJS, porque esas tablas no pueden tener RLS). Todo lo demás depende de RLS real, verificada además por una suite de pruebas dedicada que corre como app_user sin BYPASSRLS , cada test dentro de una transacción con ROLLBACK .

1.4 Estado de implementación por funcionalidad

Funcionalidad	Estado
Autenticación (cookies httpOnly + refresh + revocación de sesión)	Completa
RLS multi-tenant por secretaría + rango + confidencialidad	Completa
Publicaciones (flujo de aprobación + notificaciones + SLA)	Completa
Agenda / Reuniones / Tareas / Proyectos (CRUD + RLS + tiempo real)	Completa
Documentos adjuntos (bytea in-row)	Completa, con deuda técnica reconocida por el propio código
Despacho (instrucciones, validación, evidencia, bitácora, SLA)	Completa
Comunicación (cobertura de eventos)	Completa v1, sin tiempo real todavía
"Mi bandeja" agregada	Completa
Auditoría con filtros/resumen/export CSV	Completa
Observabilidad (Sentry opcional, request-id, logs estructurados)	Completa, apagable
Notificación por email/SMS/push	No implementada — solo campana in-app
Recuperación de contraseña autoservicio	No implementada — reseteo manual por admin
Object storage para adjuntos/evidencias	No implementada (documentada como pendiente)
TLS verify-full en producción	No implementada (documentada como pendiente)

Ningún módulo "Completo" está a medio implementar en el sentido de tener rutas que no respondan o tablas sin usar. La deuda técnica reconocida (almacenamiento in-row, TLS autofirmado) está documentada en el propio código, no fue inventada por esta revisión — una señal positiva de disciplina de ingeniería.

2. Arquitectura del sistema y catálogo de módulos

2.1 Vista general

Monorepo con dos aplicaciones y una base de datos, sin backend-for-frontend ni gateway intermedio: el navegador habla directo con la API.

```
apps/  
  api/   NestJS 10 + TypeScript, sin ORM (pg / node-postgres crudo)  
  web/   Next.js 16 (App Router) + React 19 + Tailwind 4  
db/  
  migrations/ esquema versionado a mano (26 archivos, sin tool de migración)  
  seeds/     datos de ejemplo (solo secretarías + roles)  
infra/  
  docker/   Postgres 16.4, y dos docker-compose (solo-DB / stack completo)
```

Por qué no hay ORM **INFERENCIA CON EVIDENCIA FUERTE**: un utilitario mapea códigos SQLSTATE de Postgres (42501 RLS, P0001 trigger) directo a excepciones HTTP; cada service escribe SQL parametrizado a mano. Tiene sentido con el resto de la arquitectura: la autorización vive en políticas RLS y triggers, no en un modelo de objetos — un ORM que abstraiera eso obligaría a reimplementar la misma lógica dos veces o a pelear contra el ORM para no hacerlo.

AGENDA.GOBER — Diagrama de contenedores (C4 nivel 2)

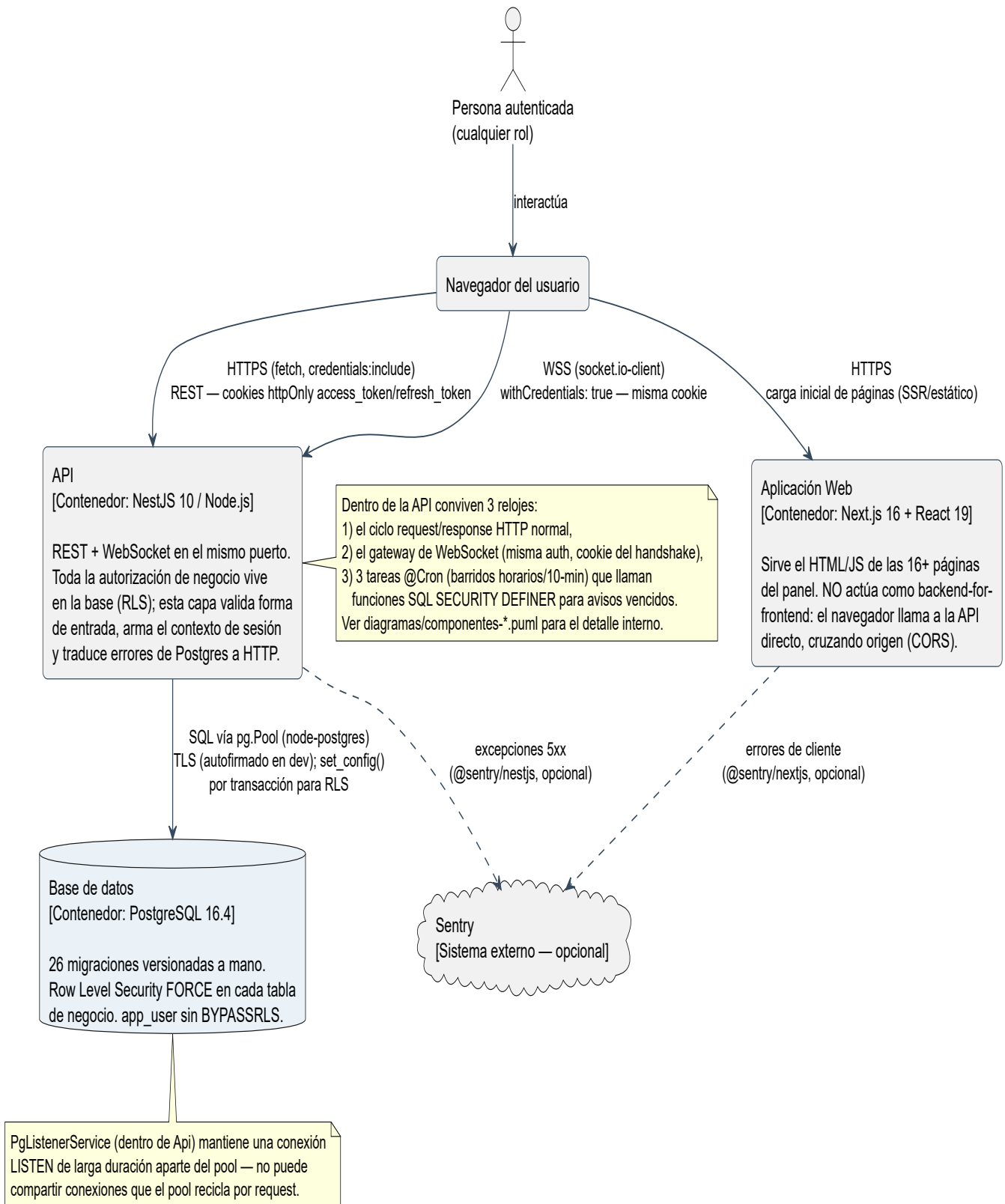


Diagrama C4 — Contenedores: el navegador habla directo con la API (no pasa por el servidor de Next.js), autenticado por cookie, con WebSocket sobre el mismo origen.

2.2 Ciclo de vida de una petición HTTP

1. El navegador hace `fetch()` con `credentials: "include"`. La sesión vive en dos cookies `httpOnly`: `access_token` (JWT, 60 min) y `refresh_token` (opaco, 30 días, `path=/auth`).
2. `helmet()` agrega cabeceras de seguridad; `cookieParser()` decodifica las cookies.
3. `JwtAuthGuard` (global) verifica el JWT salvo rutas `@Public()`. Si es válido, vuelve a consultar la base (sesión viva + no revocada + usuario activo) y arma `req.user` con el rol/secretaría **actuales** — una baja o cambio de rol se aplica en el siguiente request, no al vencer el token.
4. `TenantContextInterceptor` (global) abre una transacción (`BEGIN`) y ejecuta `set_config('app.current_rol', ...)` / `..._secretaria_id` / `..._user_id` parametrizado, dejando el cliente en un `AsyncLocalStorage`.
5. El service nunca toca el pool directamente — pasa siempre por `TxService.query()`. Si algo intentara saltarse esto, se lanza una excepción en vez de correr la consulta contra el pool "pelado" (que dejaría a RLS sin nada que evaluar y denegaría todo silenciosamente en vez de fallar ruidoso).
6. Postgres aplica las políticas RLS de la tabla con esas variables de sesión. El resultado ya viene filtrado.
7. `COMMIT` si no hubo excepción, `ROLLBACK` si la hubo.
8. Errores de Postgres se traducen: `42501 / P0001` → `403 Forbidden` con el mensaje que ya trae la base.

2.3 Tiempo real (WebSocket)

Una sola conexión Socket.IO por sesión de navegador, autenticada con la misma cookie `httpOnly` — no hay un token separado para el socket. Cada tabla con tiempo real dispara `pg_notify(canal, {id, accion})` desde un trigger — **nunca el contenido**, solo el id y si fue INSERT/UPDATE/DELETE. `PgListenerService` mantiene una conexión `LISTEN` propia y de larga duración; al recibir una notificación, **por cada socket conectado** vuelve a pedir esa fila con el contexto de sesión de ese usuario particular. Si RLS bloquea la fila para ese usuario, no le llega nada. El filtro de tiempo real es la misma política RLS, no una reimplementación en TypeScript que se pueda desincronizar.

Canales activos hoy: `publicaciones_cambios`, `eventos_cambios`, `tareas_cambios`, `proyectos_cambios`, `compromisos_cambios`, `instrucciones_cambios`, `notificaciones_cambios`. **Comunicación (cobertura) no tiene canal todavía** — es el único módulo de contenido sin tiempo real.

2.4 Tareas programadas (sweeps)

Servicio	Frecuencia	Qué hace
<code>DespachoSweepService</code>	cada 10 min	Recalcula instrucciones vencidas; SLA de acuse (recordatorio a Gabinete → escalamiento al Gobernador)
<code>PublicacionesSweepService</code>	cada hora	Recordatorio/escalamiento de publicaciones estancadas en revisión; recordatorio de aprobadas sin publicar
<code>VencimientosSweepService</code>	cada hora	Avisos de "vence pronto"/"vencida" para tareas y compromisos, con escalamiento

Los tres siguen el mismo patrón: la lógica vive en una función SQL `SECURITY DEFINER` (corre bypassando RLS, porque debe poder avisar a gente que no disparó el barrido); el servicio de NestJS solo la agenda y toma un `pg_try_advisory_lock` distinto por barrido — así, con más de una instancia de la API, solo una ejecuta cada barrido.

2.5 Archivos, caché y validación

No hay almacenamiento de objetos externo: los adjuntos se guardan como `bytea` dentro de la fila de Postgres, a propósito, para heredar la RLS de su padre sin necesitar una política propia de descarga. Tope actual: 10 MB por archivo. No hay capa de caché (Redis ni in-memory de aplicación); lo más cercano es el *rate limiting* de `@nestjs/throttler` (100/min global, 5/min en login, 20/min en refresh), con almacenamiento en memoria del propio proceso.

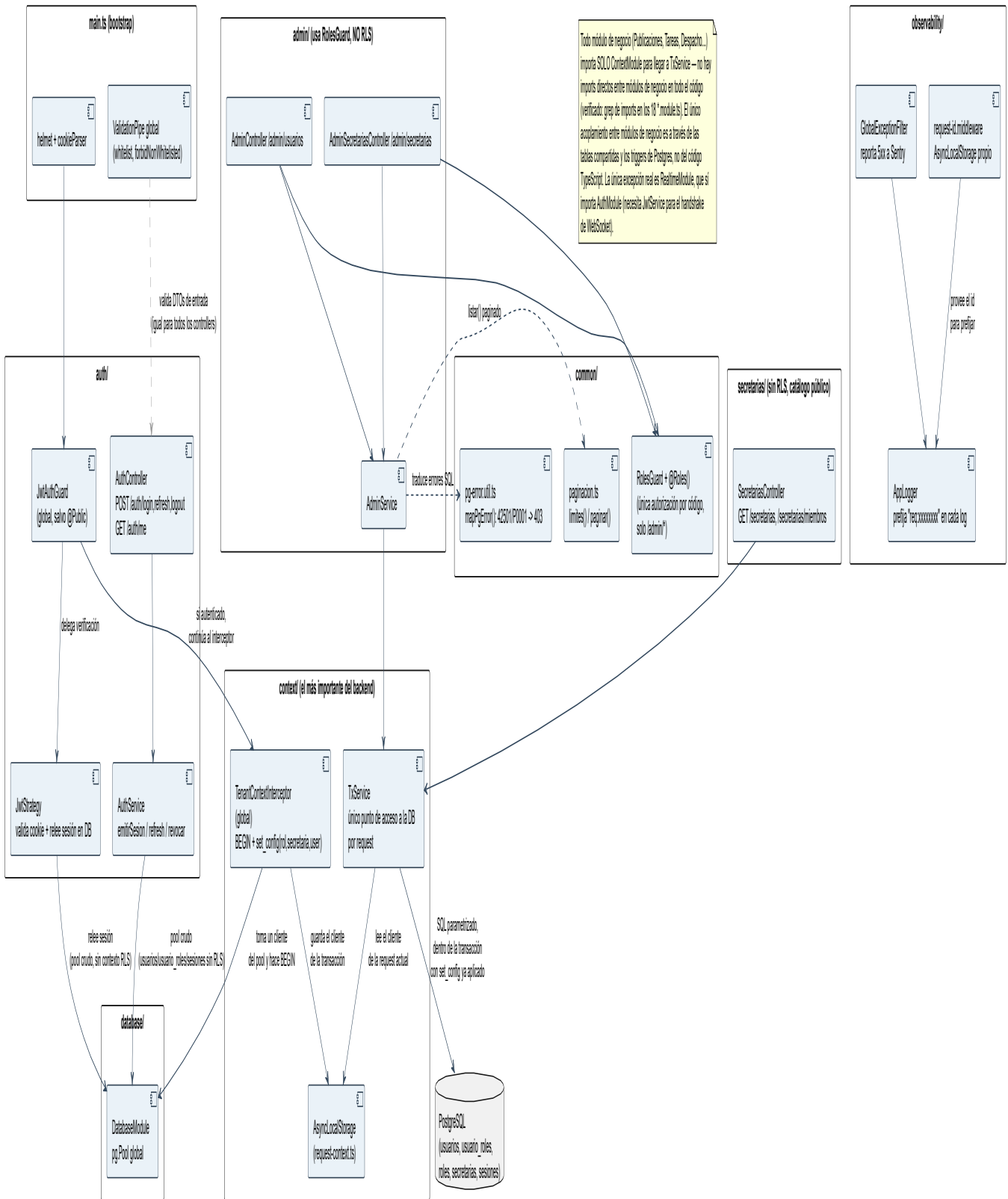
Capa	Qué valida	Ejemplo
<code>class-validator</code> (DTOs)	Forma y tipo de la entrada HTTP	<code>@IsEmail()</code> , <code>@IsUUID('4')</code>
<code>ValidationPipe</code> global	Rechaza campos no declarados (<code>forbidNonWhitelisted</code>)	Evita que el cliente mande <code>secretariaId</code> para suplantar otra secretaria
RLS (Postgres)	Quién puede ver/crear/editar/borrar cada fila	Casi todo el contenido de negocio
Triggers <code>BEFORE UPDATE/INSERT</code>	Reglas que RLS no puede expresar (OLD vs NEW, máquinas de estado)	<code>fn_validar_transicion_publicacion</code>
<code>RolesGuard</code> + <code>@Roles()</code>	Solo <code>/admin/*</code>	<code>AdminController</code>
<code>rangoDeRol()</code> frontend	Nada real — solo oculta un botón	<code>apps/web/src/lib/roles.ts</code>

2.6 Catálogo de los 18 módulos backend

Módulo	Rutas principales	Entidades	Roles autorizados (RLS real)	Tiempo real
Auth	login, refresh, logout, me, sesiones	usuarios, usuario_rols, roles, sesiones	Público (login/refresh/logout); resto, sesión válida	—
Admin	usuarios, secretarias, reset-password	usuarios, usuario_rols, secretarias	Solo <code>admin</code> (RolesGuard, no RLS)	—
Secretarías	GET secretarias, miembros	secretarias (sin RLS)	Cualquier sesión válida	—
Publicaciones	CRUD + estado	publicaciones	Secretaría+rango vs. confidencialidad; transversal sin tope	<code>publicaciones_cambios</code>
Documentos	subir, descargar, borrar	documentos (bytea)	Hereda de la publicación padre	(vía <code>publicaciones_cambios</code>)
Agenda (Eventos)	CRUD + responsables	eventos_agenda, evento_responsables	Secretaría+rango, o invitado; editar exige director+	<code>eventos_cambios</code>
Reuniones	acta, compromisos	reunion_actas, compromisos	Igual que Agenda + responsable del compromiso	<code>compromisos_cambios</code>
Tareas	CRUD + asignados	tareas, tarea_asignados	Secretaría+rango, o asignado; editar completo exige director+	<code>tareas_cambios</code>
Proyectos	CRUD	proyectos	Secretaría+rango; editar exige director+	<code>proyectos_cambios</code>
Despacho	instrucciones (10 rutas) + items (6 rutas)	instrucciones, instruccion_items,	Solo transversales ven la instrucción; responsable ve/actúa sobre su ítem	<code>instrucciones_cambios</code>

		item_evidencias, instruccion_bitacora, vistos		
Comunicación	CRUD + equipo + gabinete-visto	cobertura	unicom+transversales gestionan; secretaría dueña solo mira	— (pendiente)
Notificaciones	listar, conteo, leer	notificaciones	Cada usuario ve/marca solo las suyas	notificaciones_cambios
Pendientes	GET /pendientes	lee publicaciones, tareas, compromisos, cobertura	Cualquier sesión (recortado por rol en el SQL + RLS de la fuente)	—
Vencimientos	solo cron	lee/escribe tareas, compromisos, notificaciones	N/A (job de sistema, SECURITY DEFINER)	—
Auditoría	listar, resumen, export CSV	auditoria (poblada por triggers de ~15 tablas)	Solo gobernador/jefe_gabinete/admin	—
Gabinete	resumen	rollup de publicaciones, eventos, tareas	Cualquier sesión (el SQL no filtra; el frontend oculta el panel)	—
Indicadores	resumen	rollup de publicaciones, tareas, proyectos	Cualquier sesión válida	—
Realtime	WebSocket (mismo puerto HTTP)	N/A (reenvía filas)	Igual que la tabla reenviada	Es el propio motor

API — Componentes del núcleo transversal (autenticación, contexto RLS, identidad)



Componentes del núcleo transversal — el **TenantContextInterceptor** es, según la propia documentación del backend, "el componente más importante del sistema".

2.7 Frontend

`app/(panel)/layout.tsx` es el shell autenticado: llama `GET /auth/me` al montar (no hay token legible por JavaScript que decodificar), redirige a `/login` si falla, y si tiene éxito monta `SessionProvider` + `RealtimeProvider` + sidebar/topbar para todas las páginas del panel. `lib/api.ts` es el cliente HTTP único: `fetch` con cookies, reintento automático de un 401 vía `POST /auth/refresh` con *single-flight* (si caen varias peticiones 401 a la vez, solo se dispara un refresh, no varios). Todas las páginas siguen el mismo patrón: `useEffect` + fetch tipado + `useState` de carga/error + (donde aplica) suscripción a `useRealtime()`.

2.8 Discrepancias encontradas entre documentación y código VERIFICADO

1. **README de la API describe un mecanismo de socket obsoleto.** Dice que el cliente se conecta con un token explícito (`auth: {token}`); el código real usa solo la cookie `httpOnly` — inconsistencia dentro del mismo documento, resto de una migración anterior de JWT-en-header a cookie.
2. **Sección "Probar" del mismo README usa `Authorization: Bearer`.** Ya no funciona — el sistema solo extrae el token de la cookie.
3. **README del frontend desactualizado en casi todo:** dice que el token vive en `localStorage` (falso, son cookies `httpOnly`); lista solo 3 secciones cuando hay 16 páginas; dice "sin manejo de expiración" y "CORS abierto" (ambos ya resueltos); menciona el puerto incorrecto.

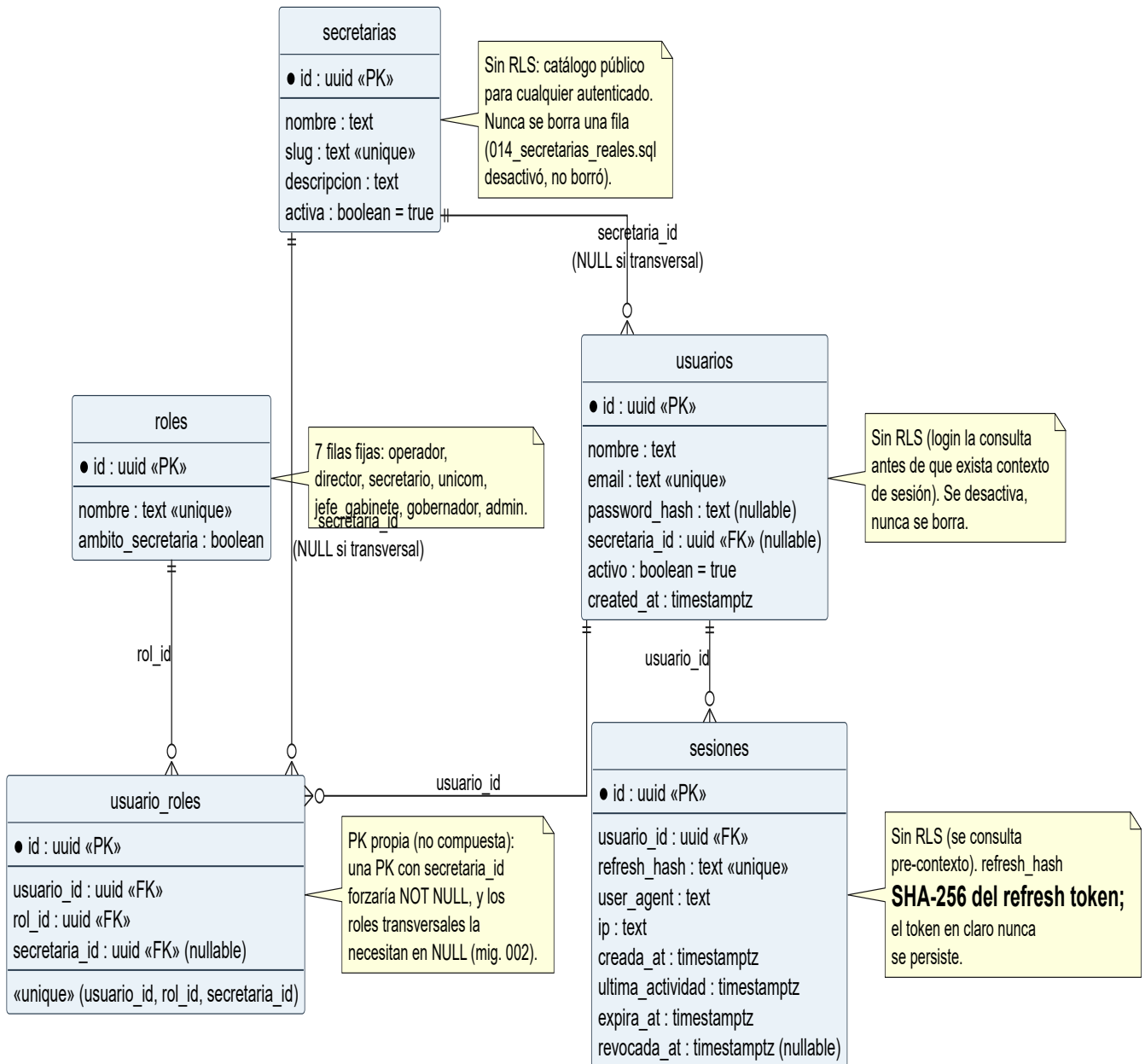
Estas tres son evidencia directa de que la documentación de los README no se actualizó al ritmo del código — un patrón a tener en cuenta al confiar en cualquier README de este repositorio sin contrastarlo.

3. Modelo de datos

PostgreSQL 16.4. Ningún `DROP TABLE / DROP COLUMN` destructivo en toda la historia de migraciones — los cambios de organigrama se resolvieron con `UPDATE` + `INSERT ... ON CONFLICT DO NOTHING`, nunca borrando filas que otras tablas ya referencian. El modelo se documenta en 3 diagramas ER agrupados por dominio de negocio (no por orden de creación), porque un solo diagrama con las ~25 tablas del sistema sería ilegible.

3.1 Identidad y organización

Entidad-Relación — Identidad y organización (quién es quién)

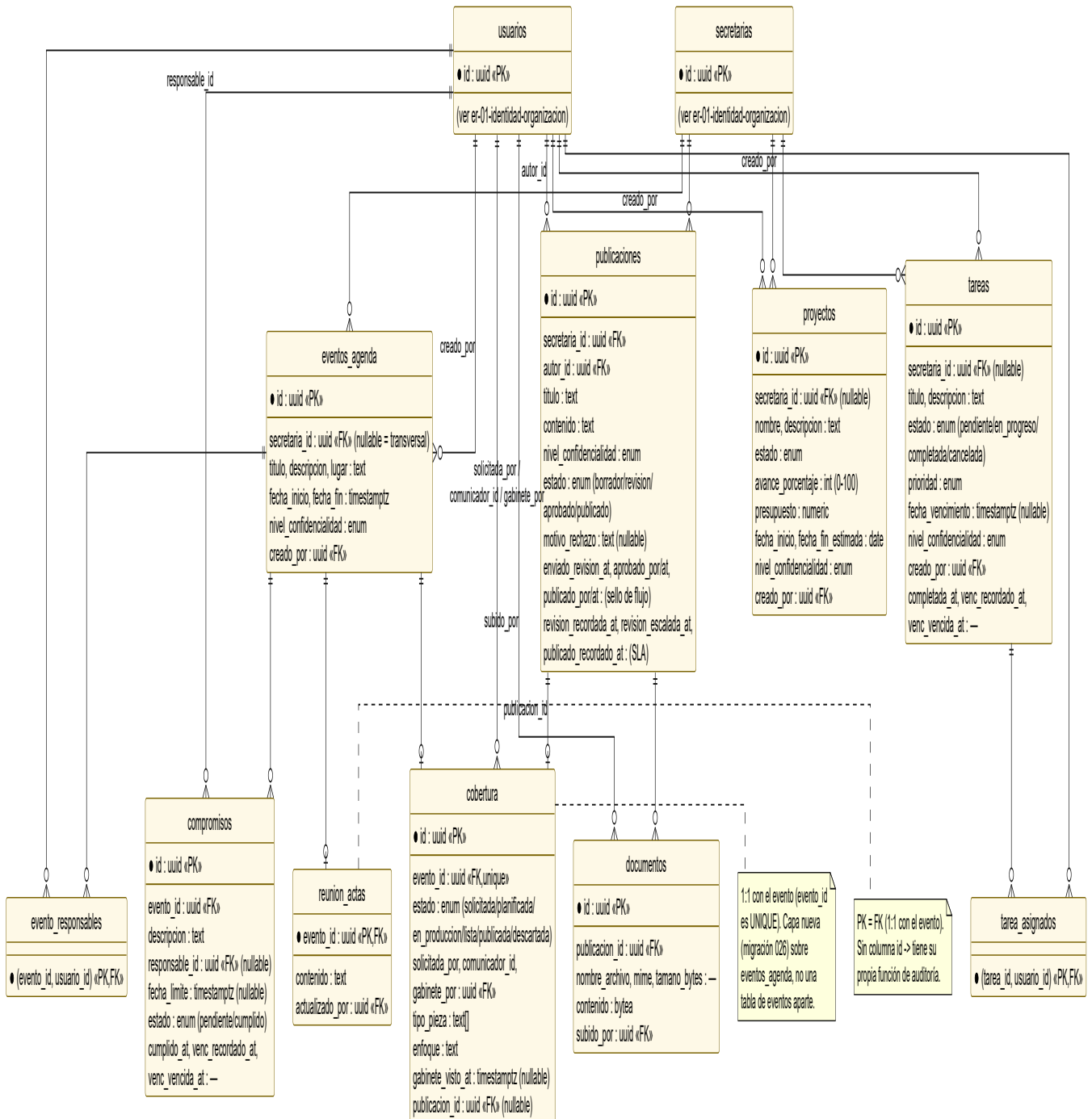


ER — Identidad y organización: secretarías, roles, usuarios, usuario_roles y sesiones.

Tabla	RLS	Notas clave
<code>secretarias</code>	Sin RLS	Catálogo público para cualquier autenticado. Las inactivas no se borran, solo dejan de listarse.
<code>roles</code>	—	7 filas fijas.
<code>usuarios</code>	Sin RLS	Las cuentas se desactivan, nunca se borran (romperían <code>autor_id</code> / <code>creado_por</code> de todo lo que crearon).
<code>usuario_roles</code>	Sin RLS	PK propia (no compuesta) porque los roles transversales necesitan <code>secretaria_id</code> NULL.
<code>sesiones</code>	Sin RLS	<code>refresh_hash</code> es SHA-256 del refresh token, nunca el token en claro.

3.2 Contenido institucional

Entidad-Relación — Contenido institucional (trabajo diario de una secretaria)



ER — Contenido institucional: publicaciones, documentos, agenda, reuniones, tareas, proyectos y cobertura.

Patrón repetido en todo este bloque: cuando una tabla necesita que alguien la vea "por estar invitado/asignado" sin pertenecer a la secretaría dueña, la vía de invitado se implementa con una función `SECURITY DEFINER` en vez de un `EXISTS` directo — porque la tabla padre también consulta la tabla de invitados para su propia vía, y un `EXISTS` en ambos sentidos genera *"infinite recursion detected in policy"*. Está documentado explícitamente como "bug real encontrado armando esto" y se repitió a propósito (mismo patrón) en 4 migraciones más.

3.3 Despacho y trazabilidad

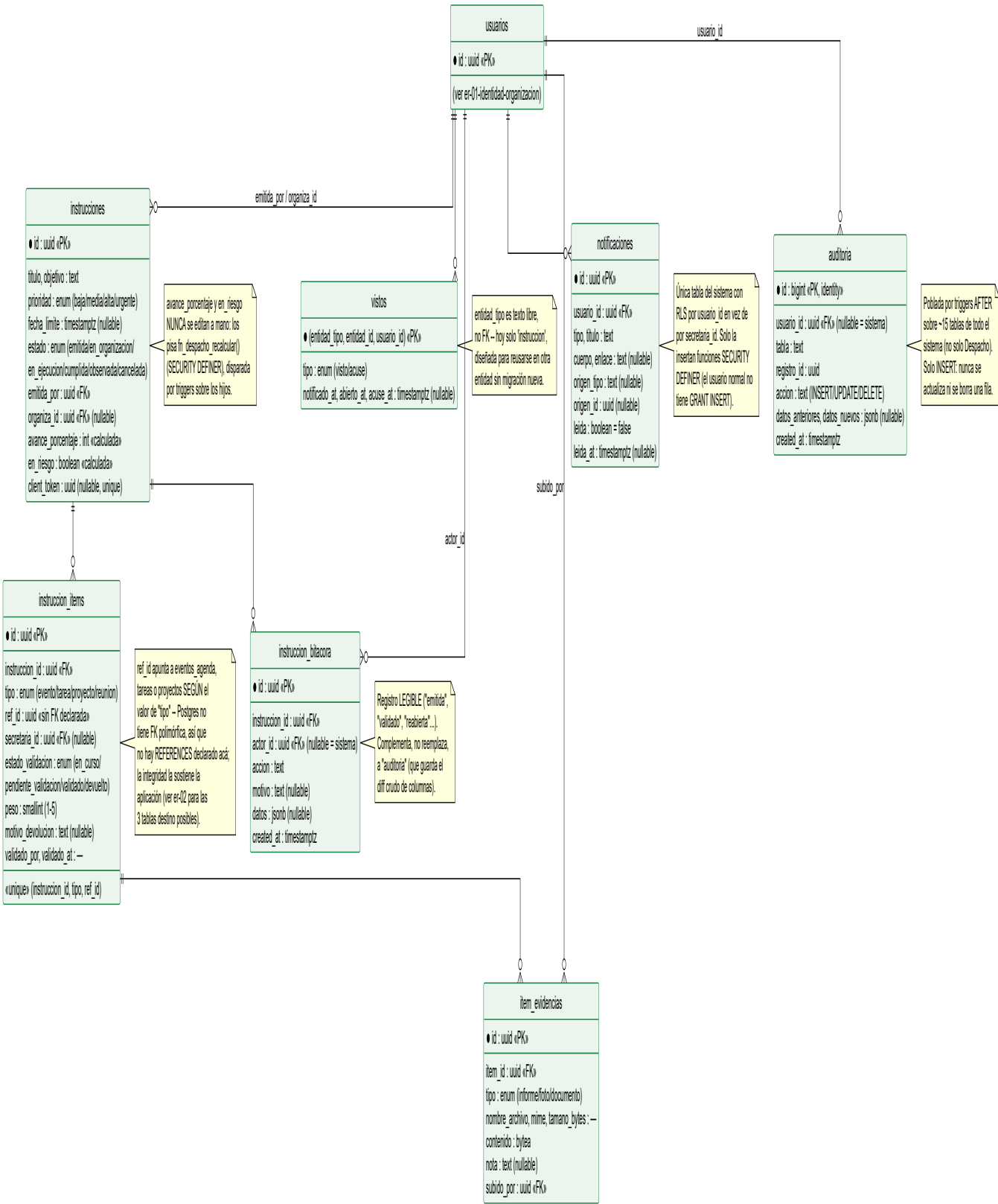
Entidad-Relación — Despacho (cadena de mando) y trazabilidad del sistema

Tabla	Notas clave
<code>instrucciones</code>	<code>avance_porcentaje</code> / <code>en_riesgo</code> son calculadas , nunca editadas a mano. <code>client_token</code> deduplica un doble clic en "Emitir".
<code>instruccion_items</code>	<code>ref_id</code> es una FK "floja" (sin <code>REFERENCES</code>) porque apunta a 3 tablas distintas según <code>tipo</code> — Postgres no tiene FK polimórfica nativa.
<code>notificaciones</code>	La única tabla del sistema con RLS por <code>usuario_id</code> , no por secretaría. Solo funciones <code>SECURITY DEFINER</code> insertan.
<code>auditoria</code>	Solo <code>gobernador</code> / <code>jefe_gabinete</code> / <code>admin</code> leen. Nunca se actualiza ni se borra — solo <code>INSERT</code> .

3.4 El patrón de automatización que se repite en 4 módulos

1. Un trigger `BEFORE UPDATE` sella una fecha (`aprobado_at` , `completada_at` ...) cuando la fila entra a cierto estado, y la limpia si sale de él o si cambia la fecha de vencimiento.
2. Un trigger `AFTER INSERT/UPDATE` , `SECURITY DEFINER` , inserta en `notificaciones` para el destinatario correcto — nunca para "todo el mundo".
3. Un barrido horario revisa lo que quedó estancado y usa una columna `*_recordado_at` / `*_escalado_at` para no repetir el aviso.

¿Por qué no hay un diagrama de clases de dominio?

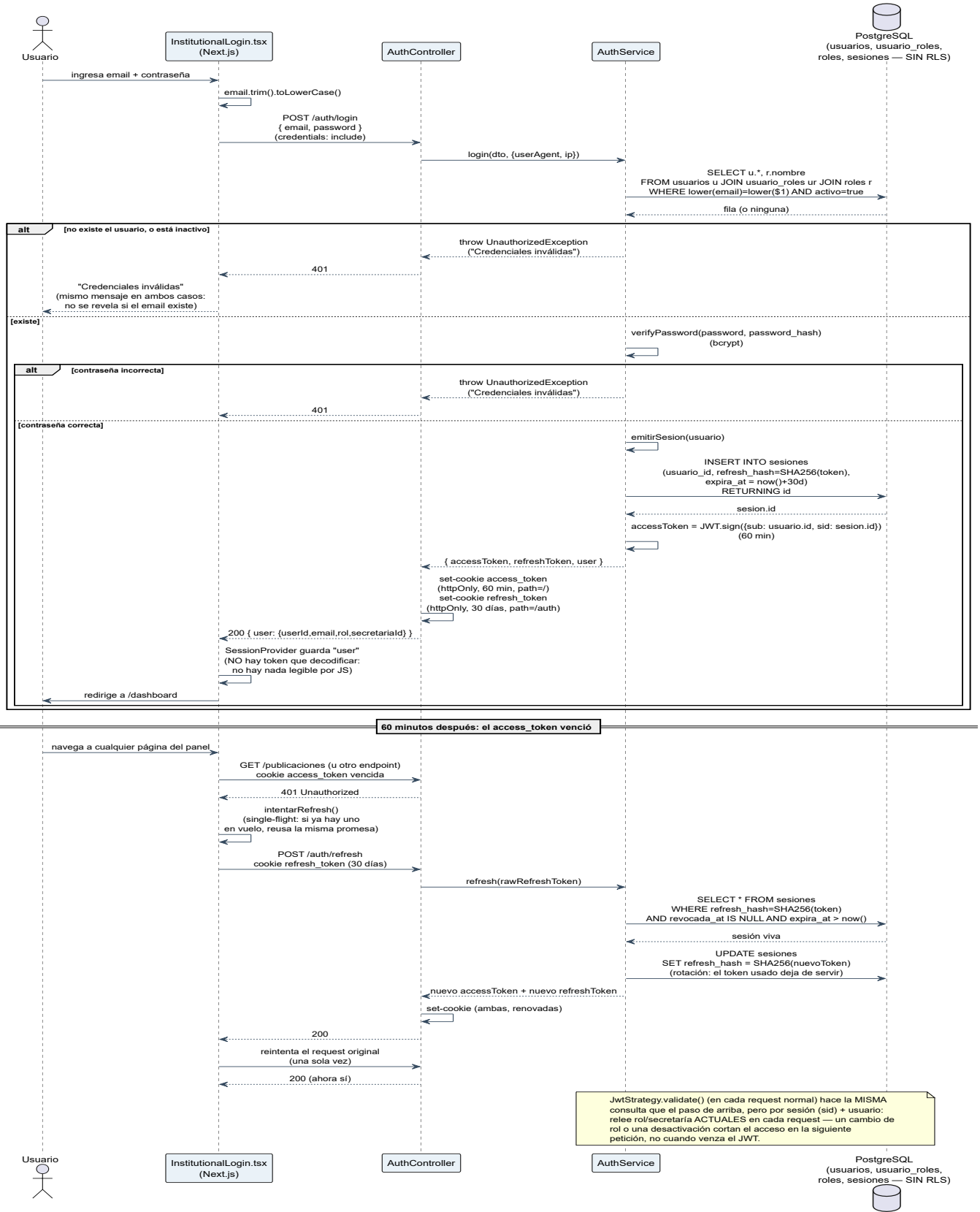
Decisión deliberada, no un olvido: este backend no tiene un modelo de objetos de dominio — no hay ORM, no hay clases `Publicacion` / `Tarea` con métodos. Cada service de NestJS es un conjunto de funciones sin estado que arma SQL; las reglas de negocio viven en funciones y triggers de PL/pgSQL. Dibujar un diagrama de clases inventaría una estructura orientada a objetos que el código no tiene. El sustituto correcto es el ER (datos) + componentes (módulos) + secuencia/actividad/estados (comportamiento).

4. Recorridos de extremo a extremo

Seis procesos reales, narrados siempre en el mismo orden: **Actor** → **Pantalla** → **Petición** → **Controlador** → **Servicio** → **Entidad** → **Respuesta** → **Actualización de la interfaz**. Se eligieron porque, entre todos, tocan cada capa del sistema al menos una vez.

1. Iniciar sesión

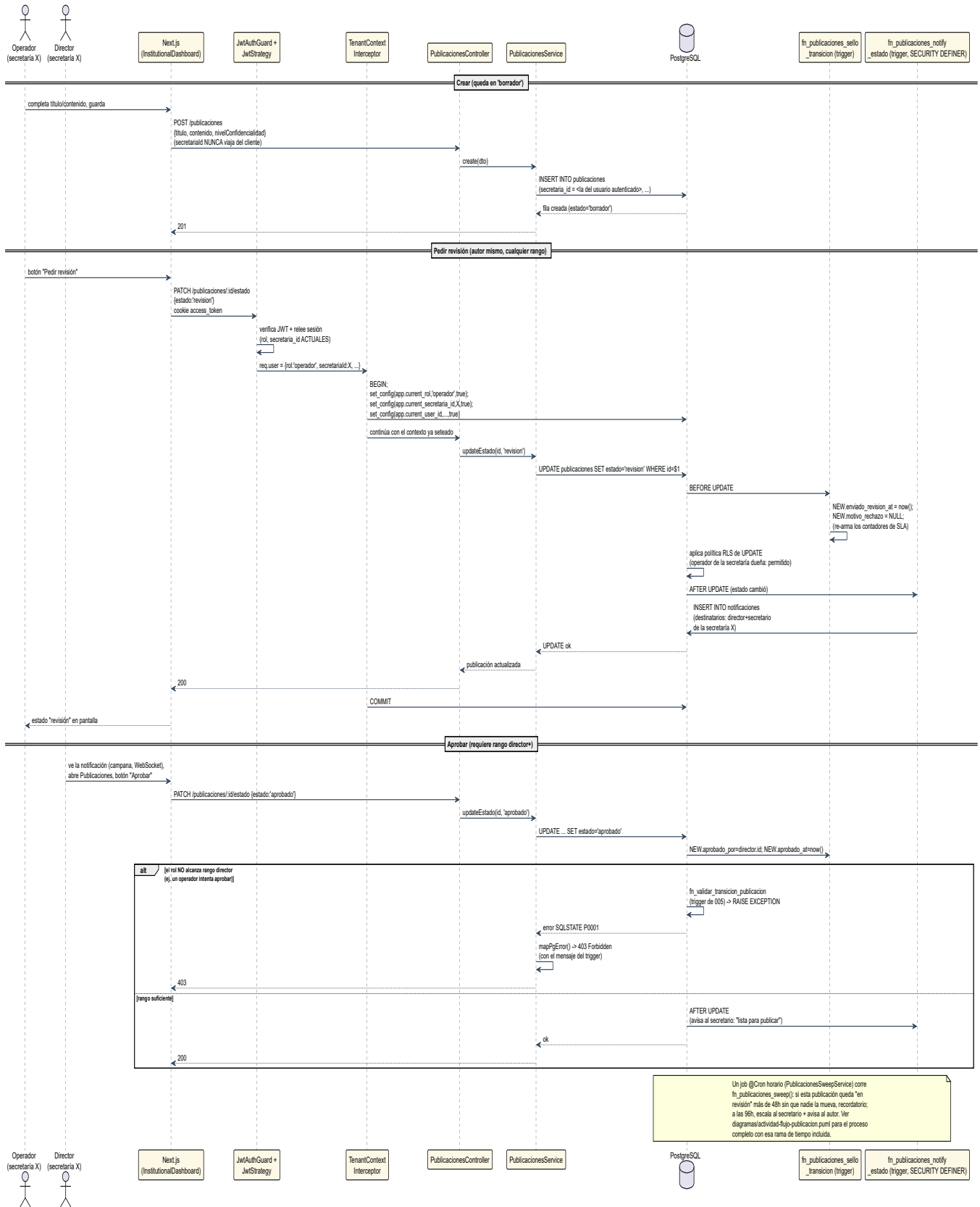
Sequencia — Login, emisión de sesión, y refresco automático de token



La consulta compara `lower(email)=lower($1)` y el DTO normaliza (`trim().toLowerCase()`) antes de validar. El mensaje de error es el mismo ("Credenciales inválidas") si el email no existe o si la contraseña es incorrecta — nunca revela cuál de las dos cosas pasó. Al vencer el `access_token`, el cliente intercepta el 401, llama una sola vez a `/auth/refresh` (single-flight) y reintenta — sin sesión cortada visible mientras el `refresh_token` siga vivo y no revocado.

2. Publicar un comunicado (con automatización)

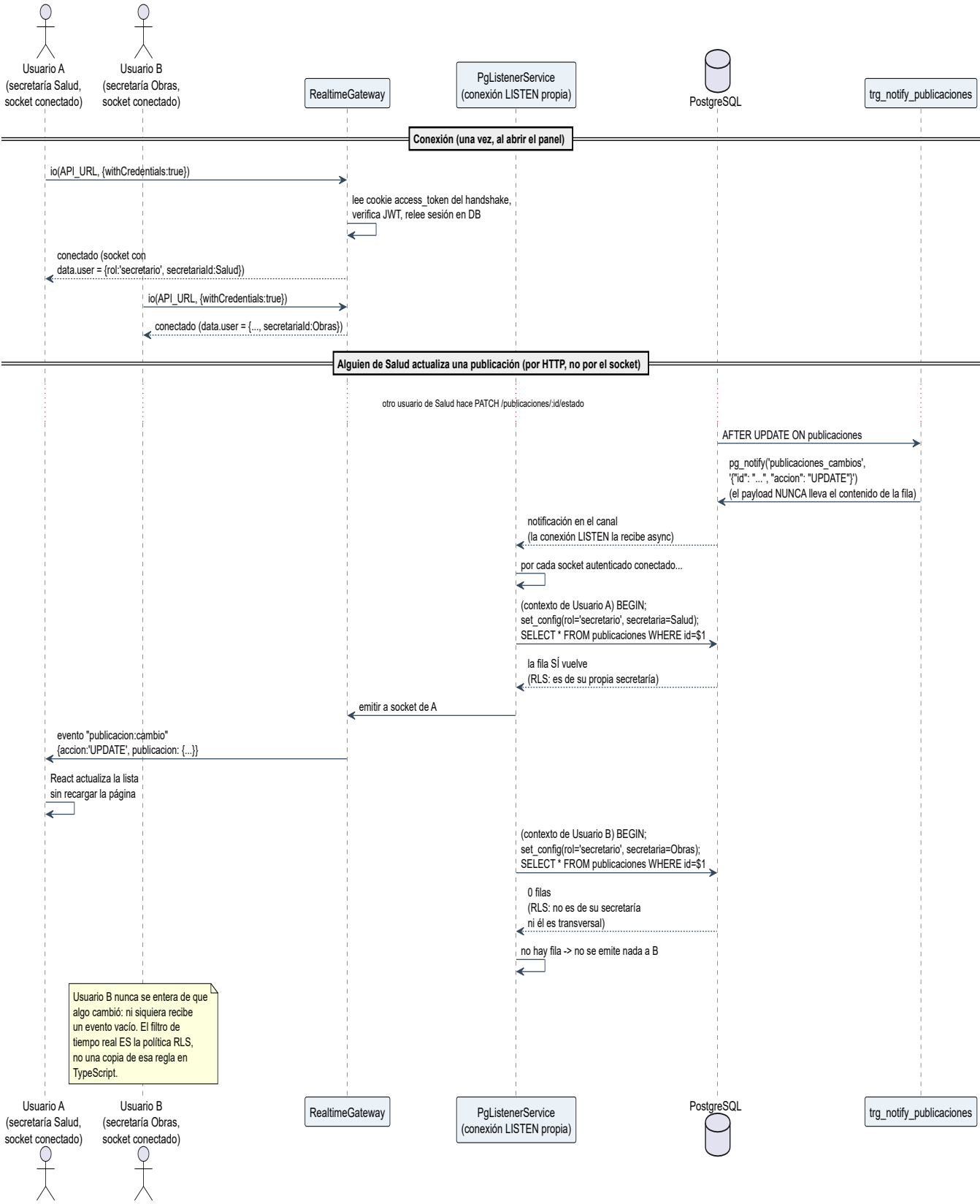
Secuencia — Publicación: crear y aprobar un comunicado (petición HTTP autenticada, capa por capa)



Nadie tiene que acordarse de avisarle a nadie: cada cambio de estado dispara, en la misma transacción, un trigger que notifica al destinatario correcto según hacia dónde se movió el estado. Si la publicación queda estancada (+48h en revisión, o aprobada sin publicar), un job horario manda un recordatorio y a las 96h escala al secretario. Ningún aviso se repite.

3. Actualización en tiempo real

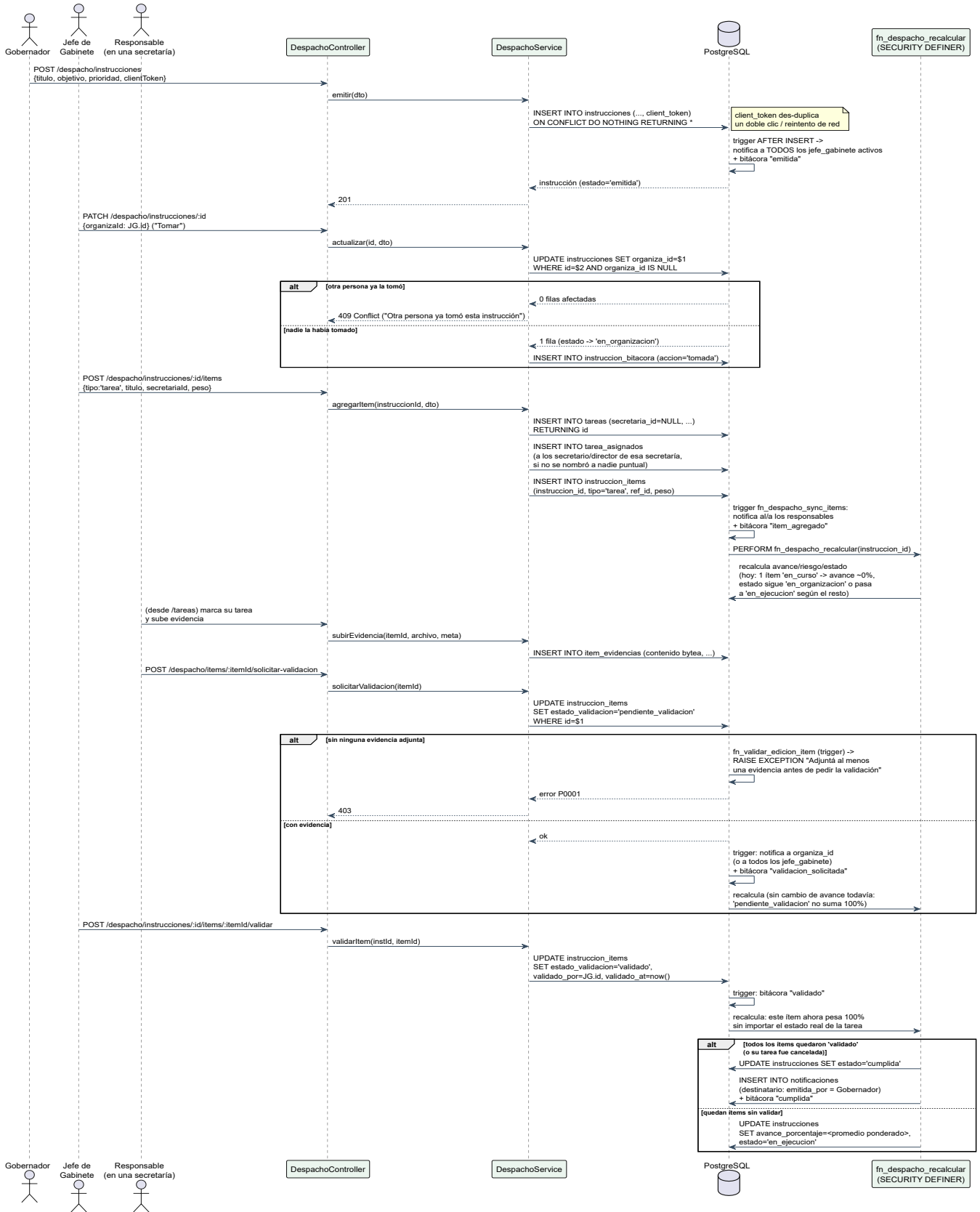
Secuencia — Propagación en tiempo real (ejemplo: dos secretarías, un cambio de publicaciones)



Vale para los 7 canales activos: el filtro de "quién se enterá" es la misma política RLS que ya protege el HTTP, vuelta a evaluar por cada socket. No existe una segunda copia de la regla de negocio en TypeScript que pueda desincronizarse.

4. Despacho: de la instrucción del Gobernador al cierre validado

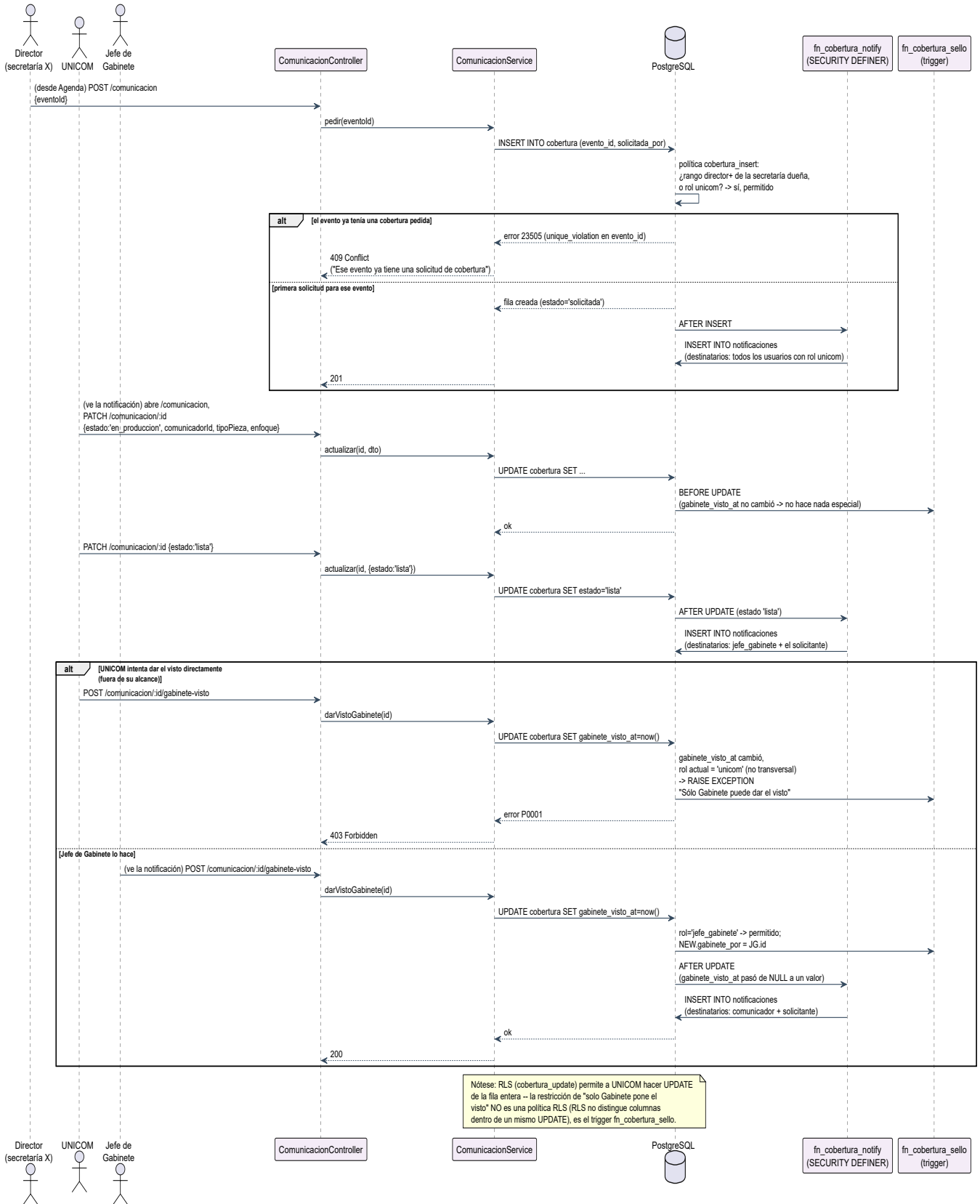
Secuencia — Despacho: emitir, desglosar, ejecutar y validar una instrucción



El recorrido con más automatización: una función recalcula avance/riesgo/estado de la instrucción cada vez que cambia un ítem — nadie edita esas columnas a mano, ni siquiera el Gobernador. El responsable de una tarea puede actuar sobre **su** ítem sin ver la instrucción completa; pedir validación exige al menos una evidencia adjunta; devolver un ítem exige motivo. Un cron cada 10 minutos persigue instrucciones vencidas y el SLA de acuse de recibo.

5. Comunicación: pedir cobertura y conseguir el visto de Gabinete

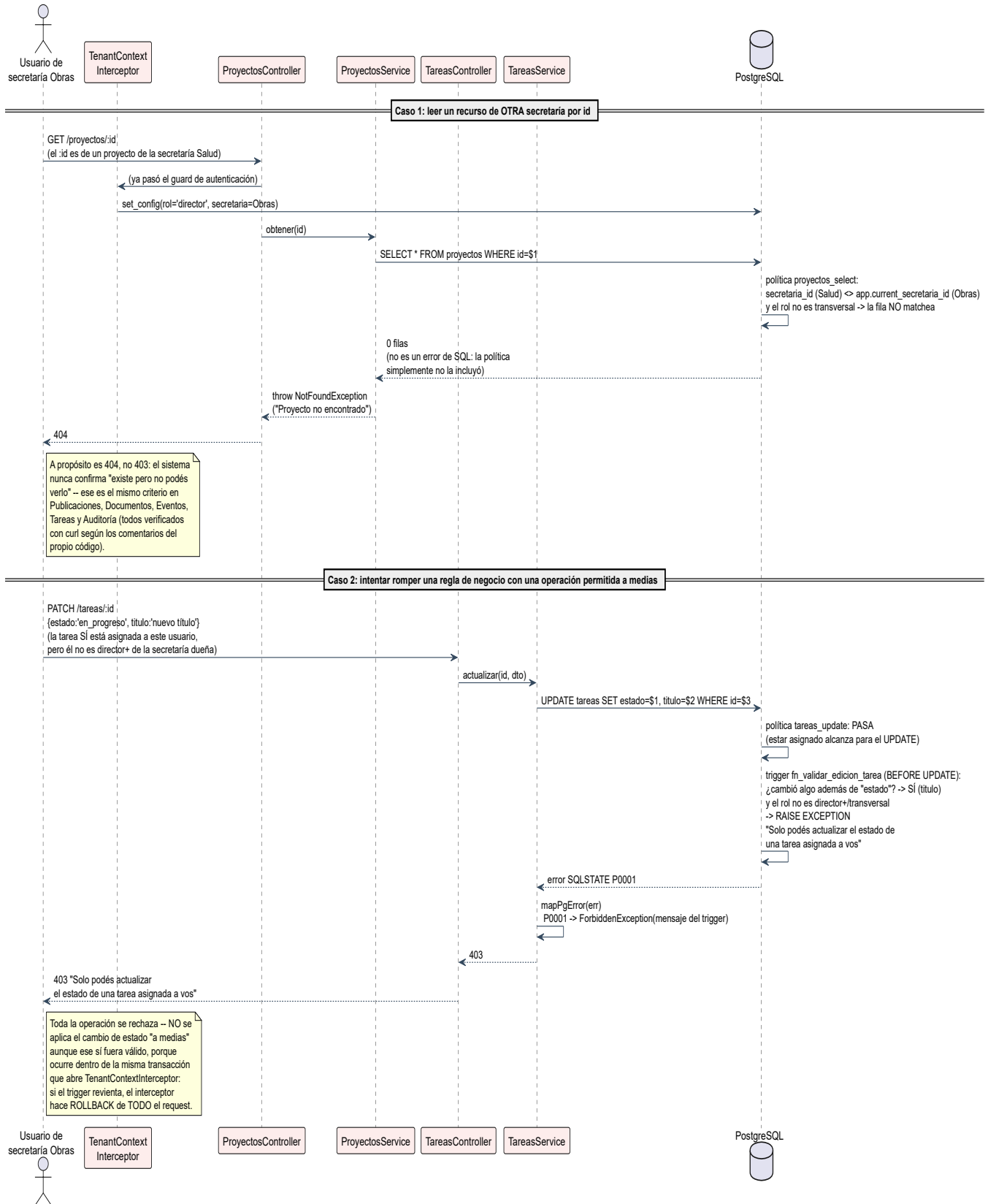
Secuencia — Comunicación: pedir cobertura, planificar y dar el visto de Gabinete



El módulo más nuevo, construido con el mismo patrón de sello+notificación, pero **sin el mismo rigor** en un punto: nadie valida a nivel de base de datos que los 6 valores de `cobertura.estado` se recorran en orden. La única transición realmente blindada por un trigger es que solo un rol transversal puede dar el visto de Gabinete.

6. Los dos caminos de "no": RLS silenciosa vs. regla de negocio violada

Secuencia — Dos formas de "no": denegado por defecto (404) vs. regla de negocio violada (403)



Leer un recurso fuera de alcance termina en **404** ("no encontrado", nunca "no autorizado" — ambigüedad intencional, no le confirma a nadie la existencia de algo que no puede ver). Violar una regla de edición (comparando fila vieja contra nueva) termina en **403** con el motivo real del trigger. Ambos mecanismos conviven en casi todos los módulos de contenido.

5. Hallazgos técnicos y plan de mejoras

Los 10 hallazgos siguientes son **PROPUESTA**: ninguno de los cambios descritos existe hoy en el código. Se separan a propósito de la descripción del estado actual, con evidencia verificada en código para cada uno.

#	Hallazgo	Impacto	Prioridad
1	TLS de base de datos sin verificación de certificado (<code>rejectUnauthorized:false</code>) en producción	Alto si API y base corren en hosts distintos; bajo mientras todo viva en <code>localhost</code>	Alta (condicional)
2	Adjuntos/evidencias como <code>bytea</code> en fila, sin object storage (tope 10 MB)	Backups más pesados; tope bajo para escaneos grandes	Media
3	Rate limiting y estado de Socket.IO en memoria del proceso (no Redis)	Con >1 instancia de API, el límite de login se multiplica y el tiempo real se pierde entre instancias	Alta (condicional)
4	Sin pruebas unitarias — toda la cobertura automatizada es e2e/RLS	Funciones puras (paginación, mapeo de errores, fechas) sin red de seguridad rápida	Media
5	Los 3 README quedaron desactualizados respecto al código	Riesgo de perder tiempo o sacar conclusiones equivocadas confiando en ellos	Baja (barato de arreglar)
6	<code>instruccion_items.ref_id</code> es FK polimórfica sin validación real	Un <code>DELETE</code> manual fuera de la app podría dejar referencias colgadas	Baja
7	<code>cobertura.estado</code> no tiene trigger de transición (a diferencia de Publicaciones/Despacho)	Vía API se puede saltar el orden de los 6 estados sin que nada lo impida en la base	Media
8	Sin autoservicio de recuperación de contraseña (reseteo manual por admin)	Fricción si la base de usuarios crece o el admin no está disponible	Baja
9	Sin documentación OpenAPI/Swagger de la API	Medio para integrar un cliente nuevo o herramienta externa	Baja
10	CI construye y prueba, pero no despliega a ningún entorno real	Ninguno hoy — no hay producción desplegada; relevante el día que exista	Sin definir

Detalle de los hallazgos más relevantes

Hallazgo 1 — TLS

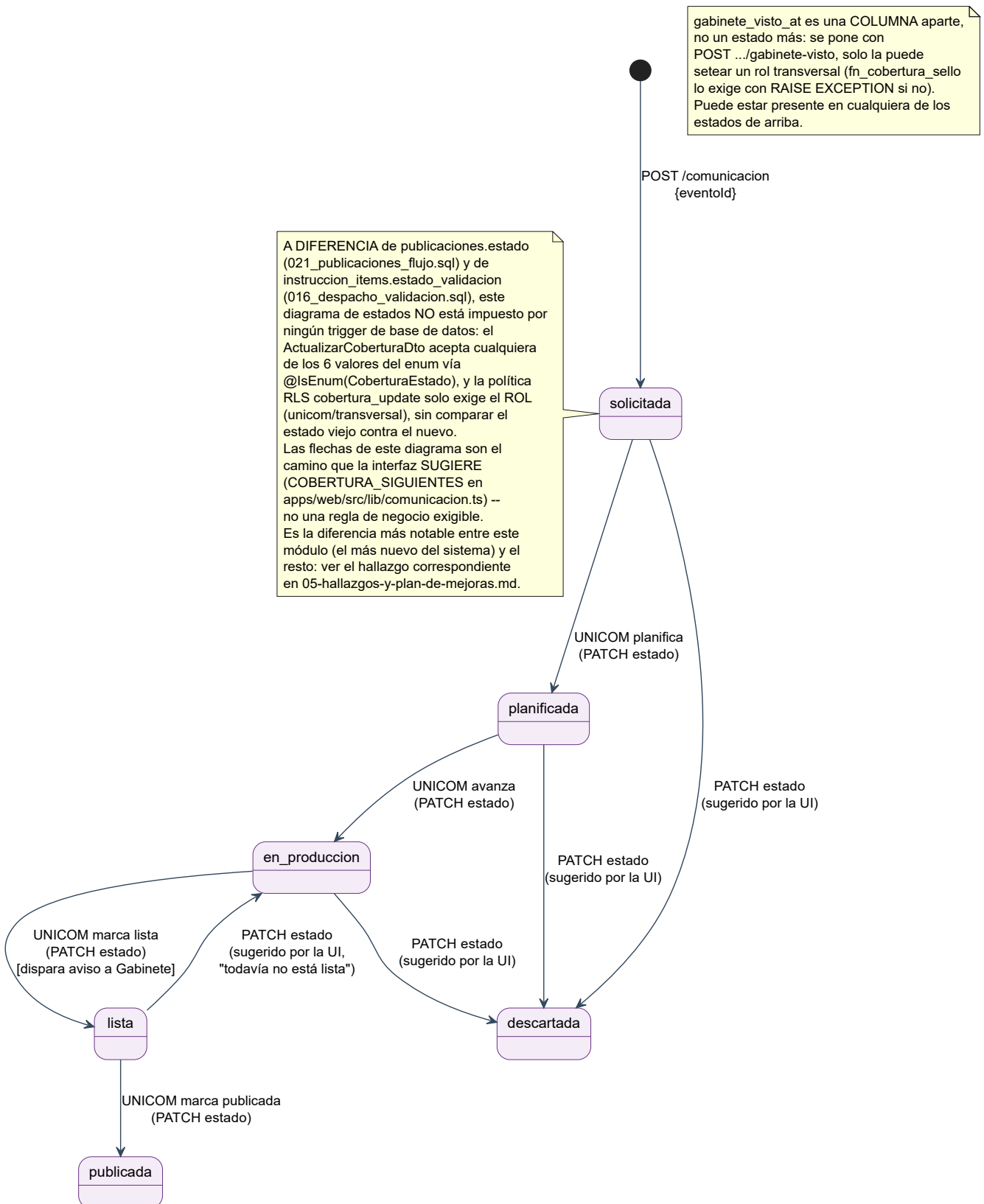
Tanto el módulo de base de datos como el listener de tiempo real conectan con `ssl:{rejectUnauthorized:false}` — cifra el tráfico pero no valida identidad del servidor (vector de MITM si la red entre API y base no es de confianza total). El propio README raíz ya reconoce la deuda: "en producción con CA propia, pasar a `sslmode=verify-full`". **Criterio de aceptación:** con un certificado auto-firmado o vencido, la conexión debe fallar al arrancar, no en silencio.

Hallazgo 3 — Redis

Hoy con una sola instancia no pasa nada. El día que se escale horizontalmente, cada instancia llevaría su propio conteo de intentos de login (5/min se vuelve, en la práctica, 5×N) y un socket conectado a una instancia nunca recibiría eventos generados en otra. **Es bloqueante** el día que se decida correr más de una instancia de la API — no antes.

Hallazgo 7 — Cobertura Hallazgo propio de esta revisión, surgido al construir el diagrama de estados de cobertura y contrastarlo contra el de publicaciones. UNICOM (o cualquier transversal) puede, vía API, mandar una cobertura de solicitada directo a publicada sin que nada lo impida en la base — solo la interfaz "sugiere" el orden. Riesgo de que el patrón se copie sin revisar para el próximo módulo nuevo, convirtiendo la excepción en norma.

Estados — cobertura.estado (Comunicación)



Único diagrama de estados del sistema sin ninguna transición impuesta por trigger — la evidencia visual del Hallazgo 7.

6. Puntos críticos — Evaluación de Dirección de Arquitectura

Todo lo anterior es la fotografía técnica del sistema, ya verificada línea por línea. Esta sección es distinta en naturaleza: es una lectura de **dirección**, construida sobre esa evidencia, enfocada en lo que un comité de arquitectura o un CTO exigiría resolver antes de confiar el sistema a la operación diaria de un gobierno departamental — continuidad de negocio, riesgo organizacional y secuenciación ejecutiva, más allá de la corrección técnica módulo por módulo que ya cubre la [Sección 5](#).

6.1 Fortalezas reales — lo que no hay que romper al seguir construyendo

1. **La autorización vive donde debe vivir.** Delegar el control de acceso a RLS de PostgreSQL en vez de a capas de aplicación es, en la experiencia de la mayoría de las auditorías de arquitectura, la decisión que más incidentes de fuga de datos evita a largo plazo — porque sobrevive a bugs de la API, a un nuevo desarrollador que se olvida de un `WHERE`, e incluso a un acceso directo a la base con las credenciales de aplicación. Que además se haya *probado* con un pentest real, y no solo diseñado en teoría, coloca a este sistema por encima del promedio de sistemas internos de gobierno de tamaño equivalente.
2. **La automatización de SLA y notificaciones reduce el factor humano como punto de falla.** Un proceso de gobierno que depende de que alguien "se acuerde" de escalar un pendiente es, en la práctica, un proceso que falla en la primera semana ocupada. Resolverlo con triggers y barridos —no con disciplina personal— es una decisión de diseño correcta y poco común.
3. **La deuda técnica está documentada por quien la creó, no descubierta por esta auditoría.** Encontrar comentarios en migraciones que dicen explícitamente "esto es un primer corte, para archivos grandes conviene migrar a object storage" es una señal de madurez de ingeniería: el equipo sabe dónde están los límites de lo que construyó.
4. **El nivel de documentación de arquitectura alcanzado (esta misma carpeta) es, en sí mismo, un activo de continuidad** — la mayoría de los sistemas de este tamaño no tienen ni un ER actualizado, y este tiene 25 diagramas validados con el compilador real, no solo dibujados a mano.

6.2 Riesgos críticos que esta evaluación agrega al informe técnico

Los 10 hallazgos de la [Sección 5](#) son correctos pero están escritos desde el código hacia afuera. Los siguientes puntos se escriben desde la **continuidad del servicio de gobierno** hacia adentro — son la pregunta que haría un director antes de firmar el "listo para producción".

R1 — Sin alta disponibilidad de PostgreSQL ni evidencia de un simulacro de restauración. Todo el sistema — agenda del Gobernador, cadena de mando del Despacho, comunicados oficiales — depende de una única instancia de Postgres en un único host (confirmado por el diagrama de despliegue: todo vive en `localhost`, sin réplica documentada). Existe un script de backup, pero **ningún documento de esta carpeta menciona haber ejecutado alguna vez una restauración real de ese backup**. Un backup nunca restaurado es una promesa, no una garantía de continuidad. Para un sistema que sostiene la

operación diaria de un gobierno departamental, esto es lo primero que cerraría antes de cualquier lanzamiento formal: RPO/RTO definidos, y al menos un simulacro de recuperación documentado.

R2 — No existe ambiente de staging ni pipeline de despliegue continuo (CD). El Hallazgo 10 ya señala que CI no despliega a ningún lado; la lectura de dirección es más dura: hoy no hay evidencia de ningún ambiente intermedio entre la máquina de un desarrollador y una eventual producción. El primer despliegue real de este sistema sería, literalmente, la primera vez que corre fuera de `localhost` — el peor momento posible para descubrir un problema de configuración, de red o de TLS (ver Hallazgo 1). Recomendación de dirección: un ambiente de staging, aunque sea mínimo, es un prerequisite no negociable antes de la primera puesta en producción real, no una mejora posterior.

R3 — Concentración de conocimiento crítico ("bus factor") en la combinación RLS + PL/pgSQL + triggers. La arquitectura elegida es, correctamente, más segura que el promedio — pero también exige una especialización poco común (SQL avanzado, funciones `SECURITY DEFINER`, prevención de recursión infinita en políticas). Si la persona o el equipo reducido que construyó esto no está disponible, el riesgo operativo de "nadie entiende por qué esta política rechaza esta fila" es alto. Un runbook operativo (no solo esta documentación de arquitectura, sino guías de "qué hacer cuando pasa X en producción") y al menos una segunda persona con capacidad de mantener las políticas RLS son, para un sistema de gobierno, un requisito de continuidad institucional, no un lujo de ingeniería.

R4 — Protección CSRF no confirmada explícitamente para un esquema de cookies `httpOnly`. El uso de cookies `httpOnly` para el JWT protege correctamente contra robo de token vía XSS, pero por sí solo **no** protege contra Cross-Site Request Forgery, que depende de atributos adicionales (`SameSite`) o de un token anti-CSRF explícito. Ninguno de los documentos de esta carpeta ni el resumen del reporte de pentest citado confirma este punto de forma explícita. Antes de considerar el sistema seguro para producción, un director de arquitectura pediría una línea explícita de verificación: qué atributo `SameSite` llevan las cookies de sesión, y si el pentest lo cubrió o quedó fuera de su alcance.

R5 — Sin rotación de secretos ni plan de respuesta a compromiso de credenciales. La convención `_FILE` para secretos (evitar variables de entorno planas) es correcta como higiene de almacenamiento, pero ningún documento describe una política de rotación de `JWT_SECRET` o de las credenciales de Postgres, ni qué pasaría operativamente si una de ellas se filtrara (¿se pueden rotar sin downtime? ¿invalida todas las sesiones activas?). Es una pregunta de gobernanza de seguridad, no solo de código.

R6 — Migraciones SQL a mano, sin herramienta de control de versión de esquema. 26 archivos secuenciales sin un `Flyway` / `Prisma Migrate` /equivalente que garantice que el mismo conjunto de migraciones, en el mismo orden, se aplicó en cada ambiente. Es coherente con la decisión de no usar ORM, pero introduce un riesgo humano real: una migración corrida manualmente en un ambiente y olvidada en otro no deja rastro verificable. Recomendaría, como mínimo, una tabla de control de versión de esquema con checksum de cada migración aplicada.

R7 — El "fan-out" de tiempo real re-consulta la base una vez por socket conectado. El diseño (reevaluar RLS por cada socket en cada cambio) es correcto en términos de seguridad — es, literalmente, lo que garantiza que el tiempo real nunca mienta. Pero tiene una implicancia de **capacidad** que no está

modelada en ningún documento: un evento que afecta a muchos usuarios a la vez (por ejemplo, el barrido de Despacho actualizando varias instrucciones en la misma corrida) puede generar un pico de `SELECT` proporcional al número de sockets conectados en ese instante. No es un problema hoy con el volumen actual de usuarios de una Gobernación; sí es una pregunta de capacidad a resolver *antes*, no después, de que el número de usuarios concurrentes crezca.

Nota de proporcionalidad. Ninguno de estos siete puntos es una falla de ingeniería — son, en su mayoría, decisiones razonables para la etapa actual del proyecto (un solo host, uso interno, volumen bajo) que se vuelven no negociables únicamente si el sistema pasa a producción real o escala. La lista existe para que esa transición no tome a nadie por sorpresa, no para desacreditar el trabajo hecho hasta acá.

6.3 Checklist de "go / no-go" antes de producción real

Preguntas concretas que este director exigiría responder, con evidencia, antes de aprobar el primer despliegue fuera de `localhost`:

#	Pregunta de gate	Relacionado con
1	¿Se ejecutó alguna vez una restauración completa del backup, de punta a punta, y cuánto tardó?	R1
2	¿Cuál es el RPO/RTO aceptado formalmente para este sistema, dado que sostiene la agenda operativa del Gobernador?	R1
3	¿Existe un ambiente de staging que replique la topología de producción, aunque sea a menor escala?	R2
4	¿Hay más de una persona en la organización capaz de escribir/depurar una política RLS o un trigger <code>SECURITY DEFINER</code> ?	R3
5	¿El pentest existente cubrió explícitamente CSRF, o quedó fuera de alcance?	R4
6	¿Hay más de una cuenta <code>admin</code> activa? (continuidad operativa de alta de usuarios/reseteo de contraseña)	Hallazgo 8 + R3
7	Si la API y la base van a vivir en hosts separados en producción, ¿está listo el certificado real para <code>verify-full</code> ?	Hallazgo 1
8	¿Se decidió ya dónde vive la producción? (bloquea CD y, en consecuencia, todo el resto del plan de salida)	Hallazgo 10

6.4 Hoja de ruta recomendada por fases

Reordenando los 10 hallazgos técnicos y los 7 riesgos de dirección por **urgencia de negocio**, no solo por esfuerzo de código:

Fase 0 — Antes de cualquier despliegue fuera de `localhost` (bloqueante)

- Simulacro de restauración de backup documentado, con tiempo medido.
- Decidir dónde vive producción (host, proveedor) — desbloquea todo lo demás.
- Confirmar explícitamente el estado de protección CSRF.
- Certificado real y `sslmode=verify-full` si API y base van a separarse de host (Hallazgo 1).

Fase 1 — Primeros 90 días de operación real

- Runbook operativo básico (qué hacer ante los incidentes más previsibles: sesión de Postgres colgada, un barrido que no corrió, un 403 inesperado de RLS).
- Corregir los 3 README desactualizados (Hallazgo 5) — barato, y es la primera fuente de error humano bajo presión.
- Trigger de transición para `cobertura.estado` (Hallazgo 7) — cierra la única inconsistencia real encontrada antes de que el módulo acumule más uso.
- Pruebas unitarias mínimas para funciones puras (Hallazgo 4).

Fase 2 — Con crecimiento de uso o de usuarios concurrentes

- Redis para Throttler y adaptador de Socket.IO, **antes** de escalar a más de una instancia de API (Hallazgo 3) — no después.
- Migrar adjuntos/evidencias a object storage (Hallazgo 2).
- Modelar el costo del fan-out de tiempo real bajo carga (R7) antes de que sea un incidente en producción.

Fase 3 — Madurez del producto

- Recuperación de contraseña autoservicio (Hallazgo 8), una vez decidido un proveedor de email.
- OpenAPI/Swagger (Hallazgo 9).
- CD real hacia el entorno elegido (Hallazgo 10).
- Segunda persona formada en RLS/PL-pgSQL, y política de rotación de secretos (R3, R5, R6).

6.5 Veredicto de dirección

El sistema está técnicamente bien construido y el modelo de seguridad es genuinamente sólido — superior al de la mayoría de los sistemas internos de gobierno de escala comparable. La decisión de anclar la autorización en RLS de PostgreSQL, verificada con un pentest real, es la elección correcta y la más difícil de deshacer bien si no se hubiera tomado desde el principio. La documentación de arquitectura alcanzada es, por sí sola, un activo poco común.

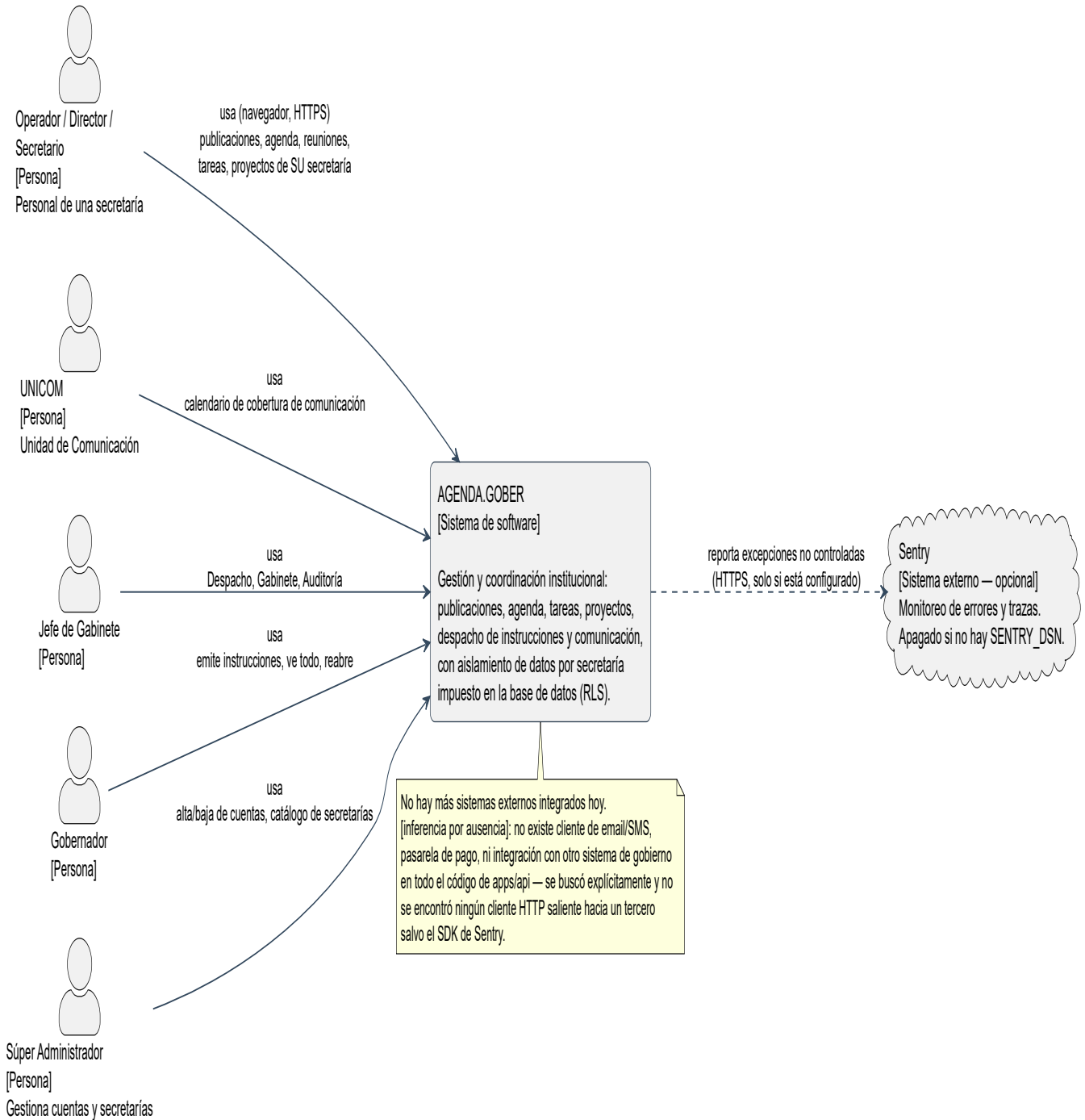
Lo que separa a este sistema de estar "listo para producción de un gobierno departamental" no es calidad de código: es **preparación operativa** — continuidad ante falla de la base de datos, un camino de despliegue probado antes del primer uso real, y reducir la dependencia de conocimiento concentrado en pocas personas. Ninguno de estos tres puntos requiere reescribir nada; todos son, ante todo, decisiones de gestión y procesos que se pueden resolver en paralelo a que el desarrollo continúe.

7. Anexo — Catálogo completo de diagramas (25)

Los 25 diagramas fuente (`.puml`, editables) viven en `docs/arquitectura/diagramas/` junto a su `.svg` ya renderizado. Se reproducen aquí completos, organizados en las mismas 8 categorías del índice original, para que este PDF sea autocontenido.

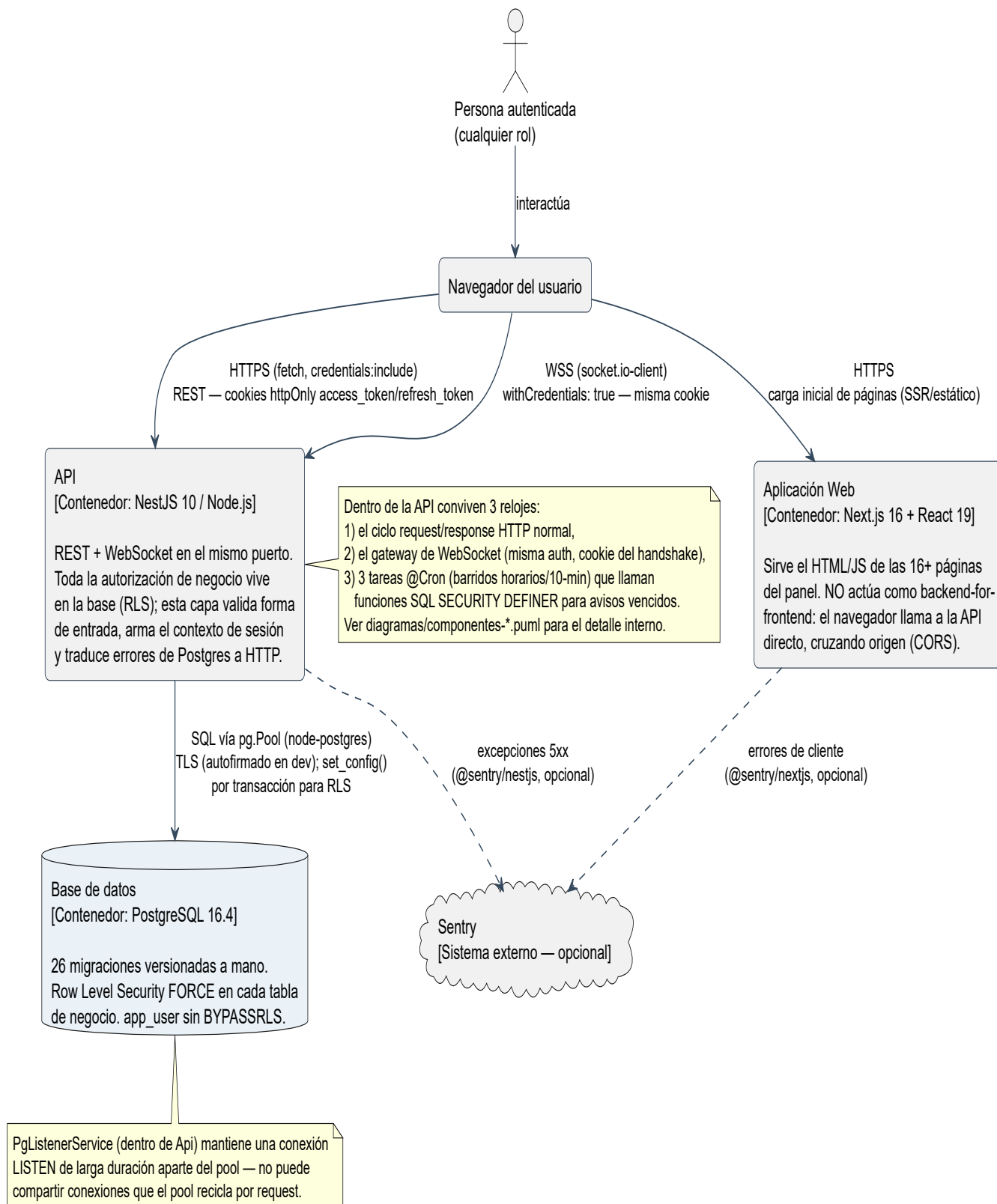
7.1 — C4: Contexto y Contenedores

AGENDA.GOBER — Diagrama de contexto (C4 nivel 1)



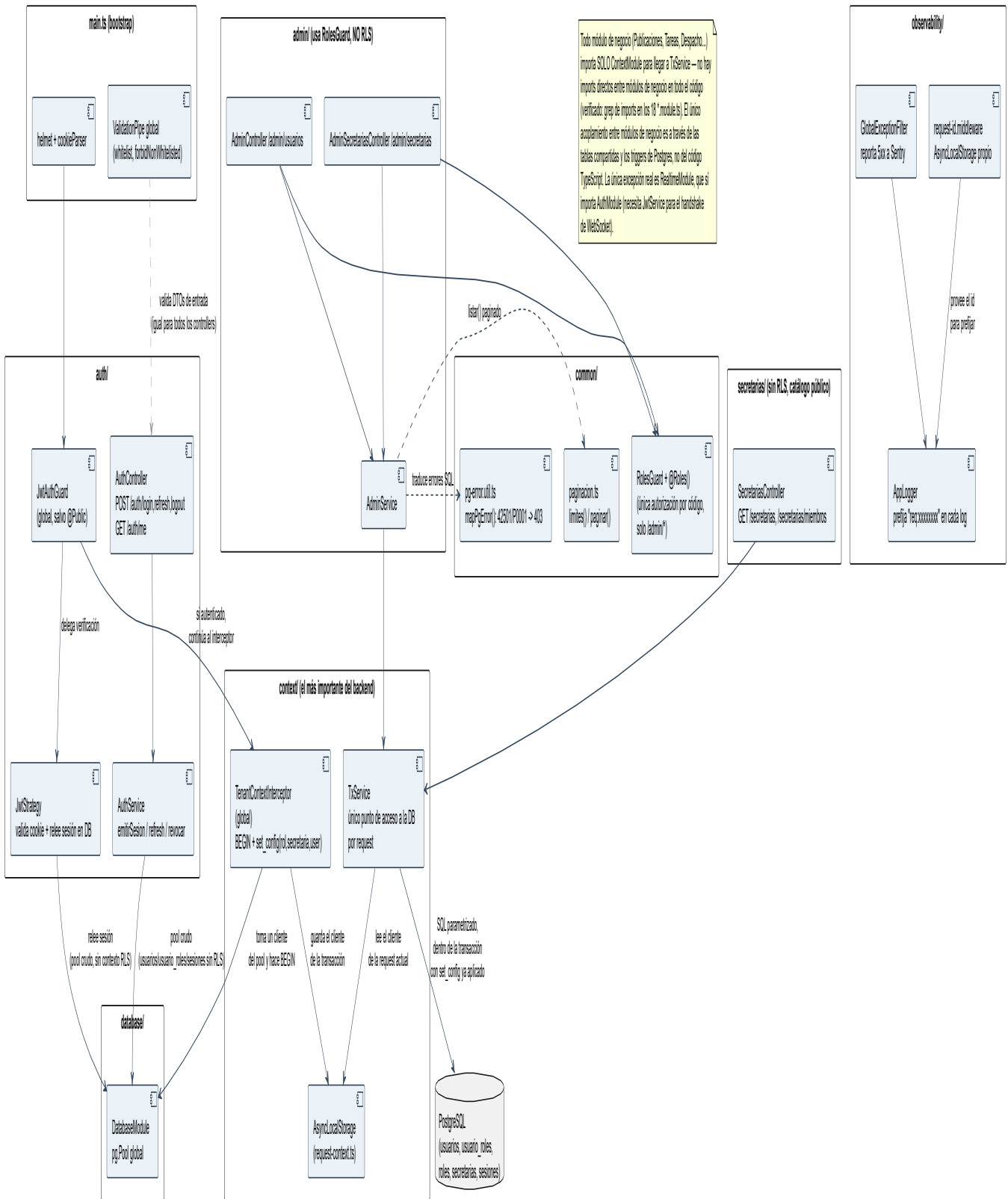
C4 — Contexto: quién usa el sistema y con qué otros sistemas habla (solo Sentry, opcional, como sistema externo real).

AGENDA.GOVER — Diagrama de contenedores (C4 nivel 2)



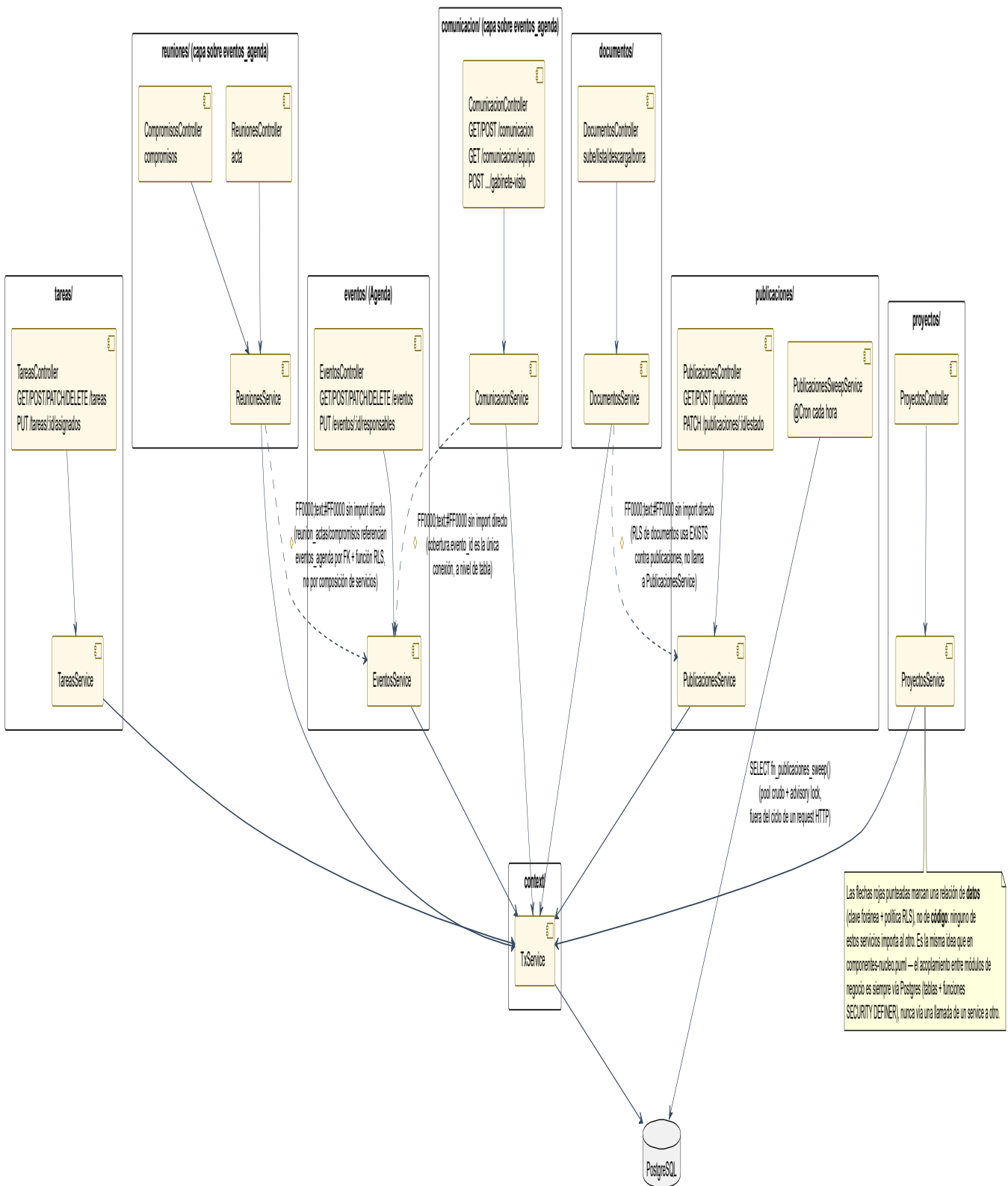
7.2 — Componentes (3, divididos por capa)

API — Componentes del núcleo transversal (autenticación, contexto RLS, identidad)



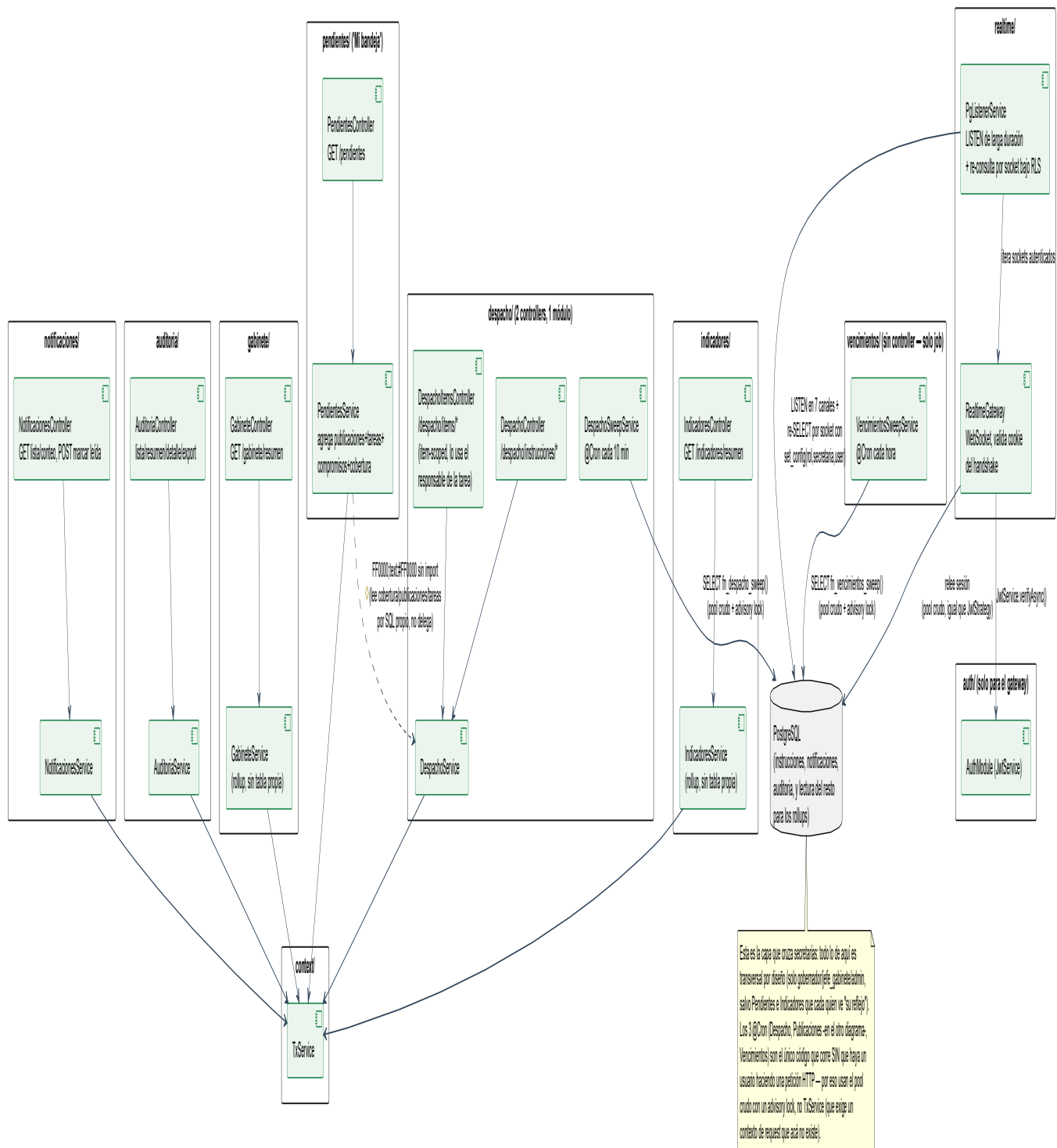
Componentes — Núcleo transversal.

API — Componentes de los módulos operativos (trabajo diario de una secretaria)



Componentes — Módulos operativos. Las flechas rojas punteadas marcan relación solo de datos (FK/RLS), no de código.

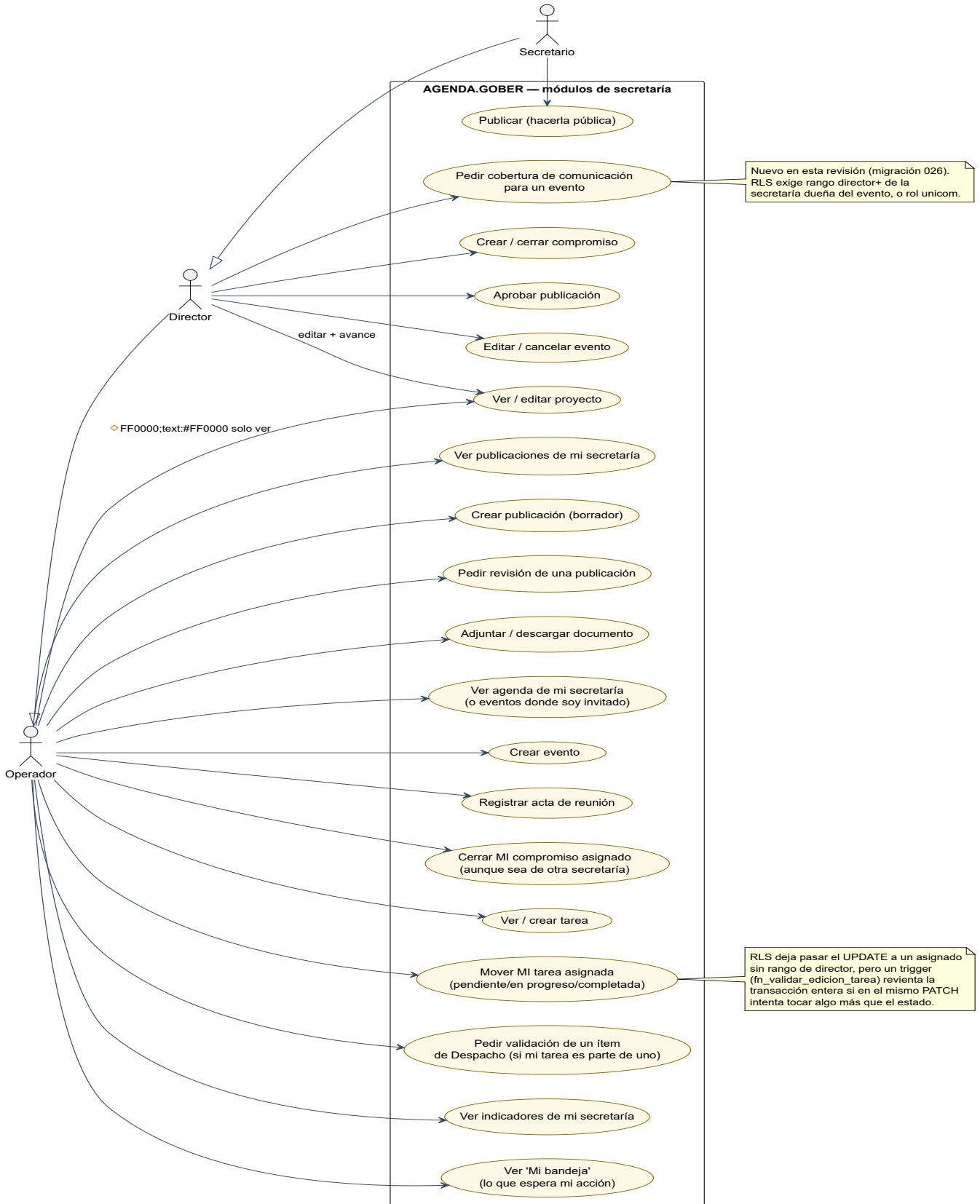
API — Despacho, agregados de conducción y automatización



Componentes — Despacho y módulos agregados.

7.3 — Casos de uso (2, por audiencia)

Casos de uso — Roles de secretaría (operador, director, secretario)

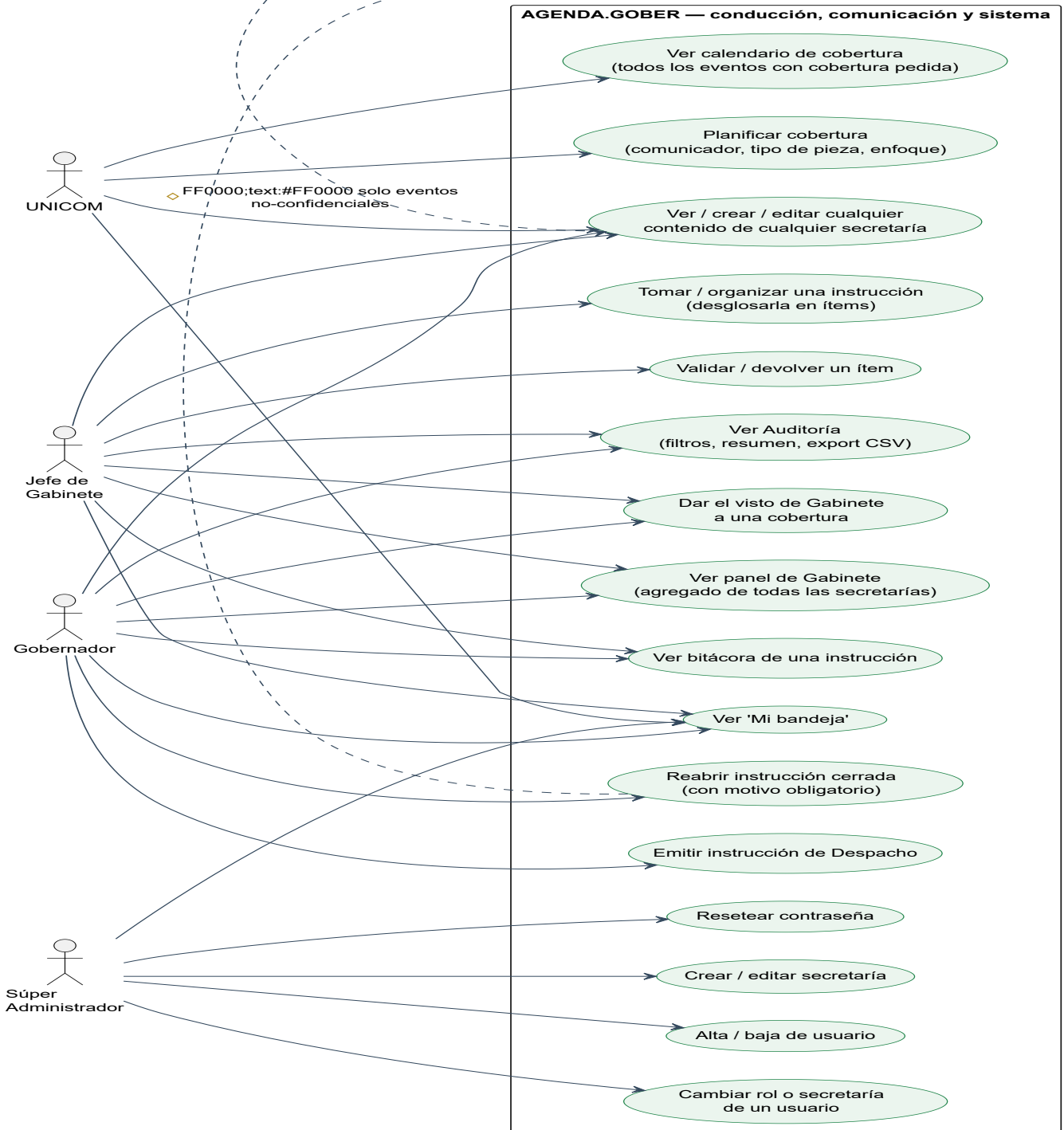


Casos de uso — Roles de secretaría (operador/director/secretario, con herencia real de `rol_rango()`).

Casos de uso — Roles transversales (UNICOM, Jefe de Gabinete, Gobernador, Administrador)

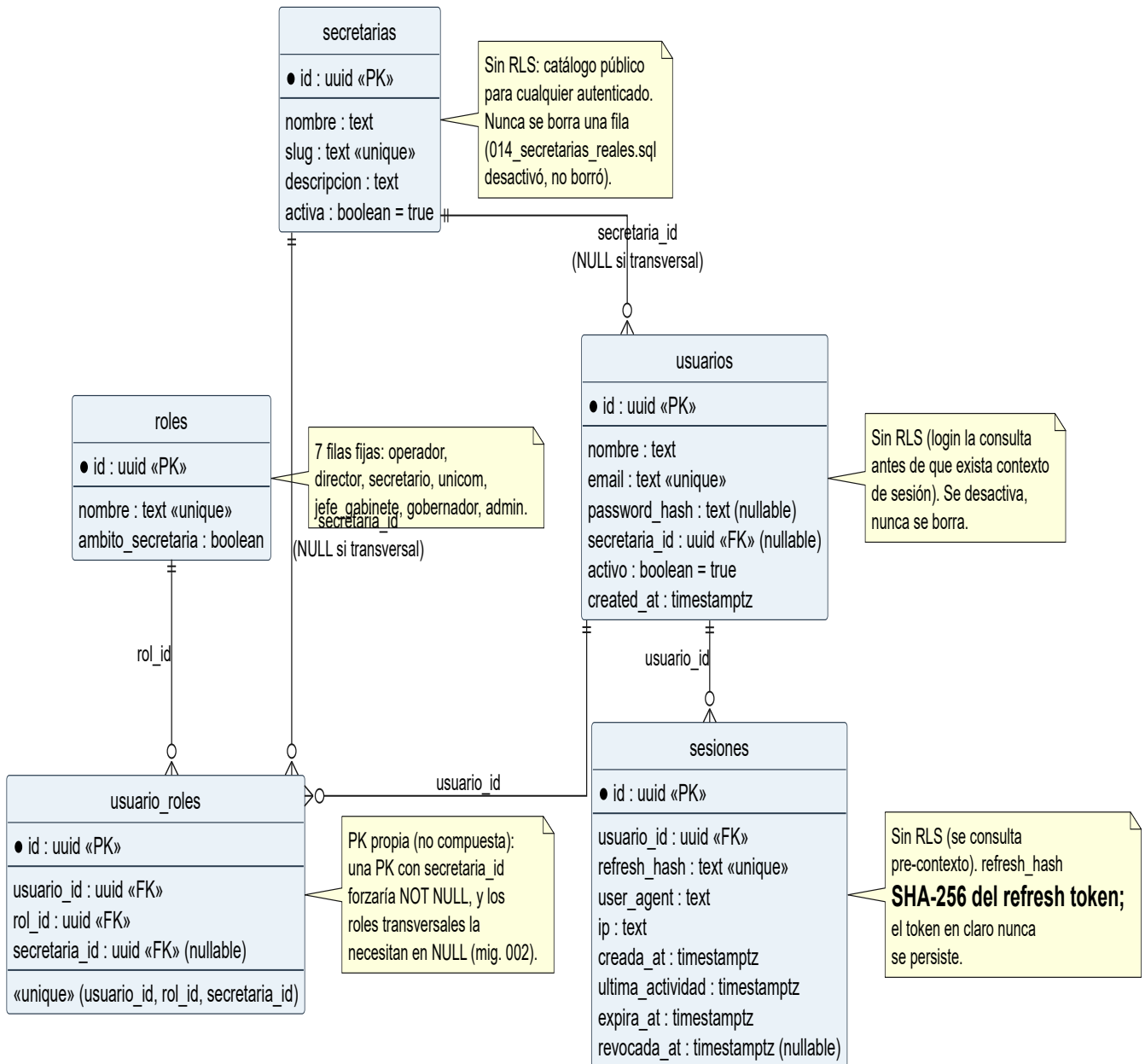
División deliberada, verificada en ambos sentidos:
 - Admin NO tiene este caso de uso: gestiona identidades pero no ve contenido estratégico.
 - Gobernador/Jefe de Gabinete NO tienen UC12-UC15: ven todo el contenido pero no administran cuentas. apps/api/README.md documenta la prueba con curl: gobernador@test.local contra GET /admin/usuarios responde 403 (RolesGuard('admin') exige el rol exacto "admin", ser transversal no alcanza).

Solo el Gobernador (chequeo explícito en DespachoService.reabrir, no solo RLS) puede reabrir.



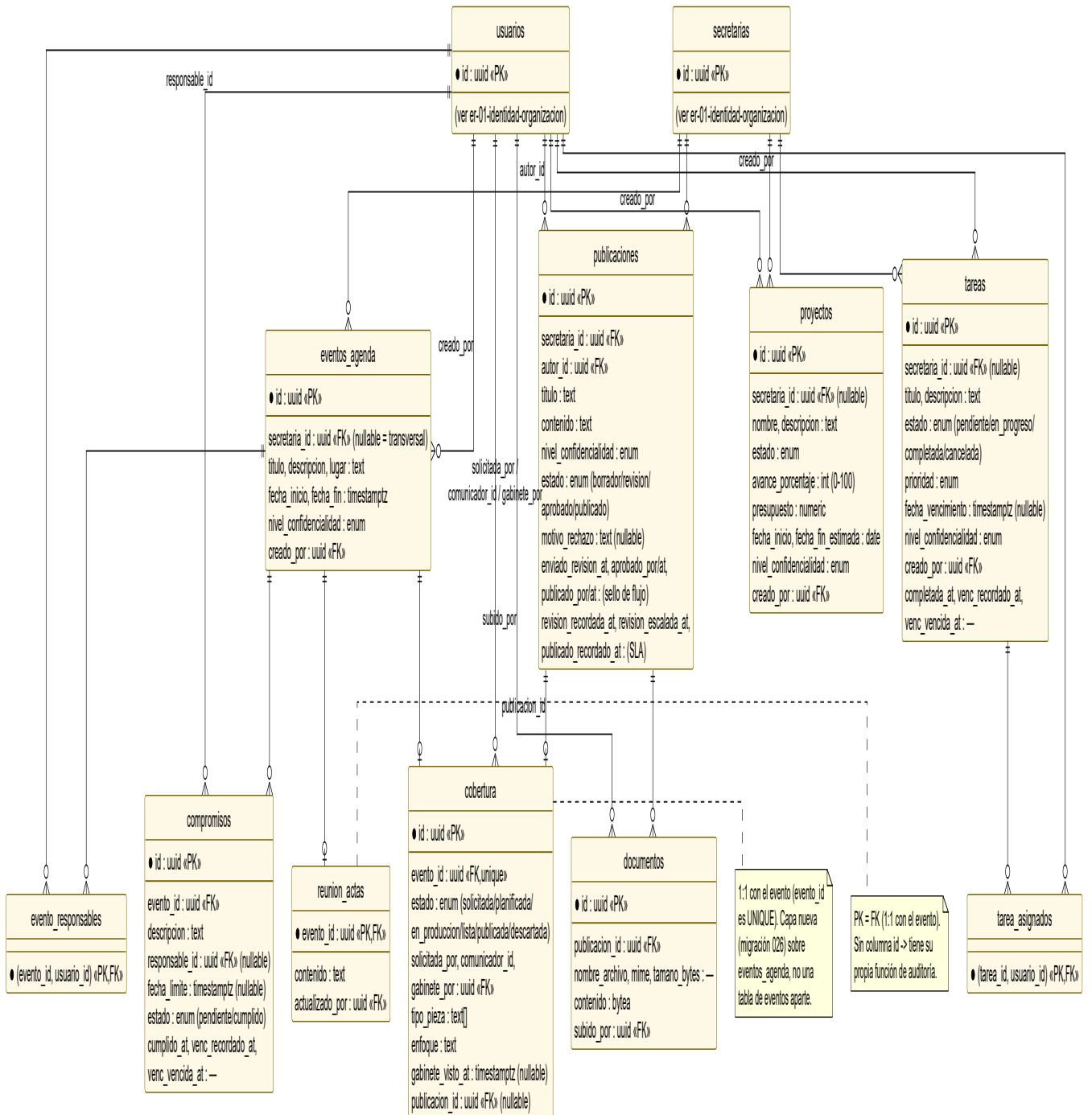
Casos de uso — Roles transversales (sin jerarquía real entre sí).

Entidad-Relación — Identidad y organización (quién es quién)

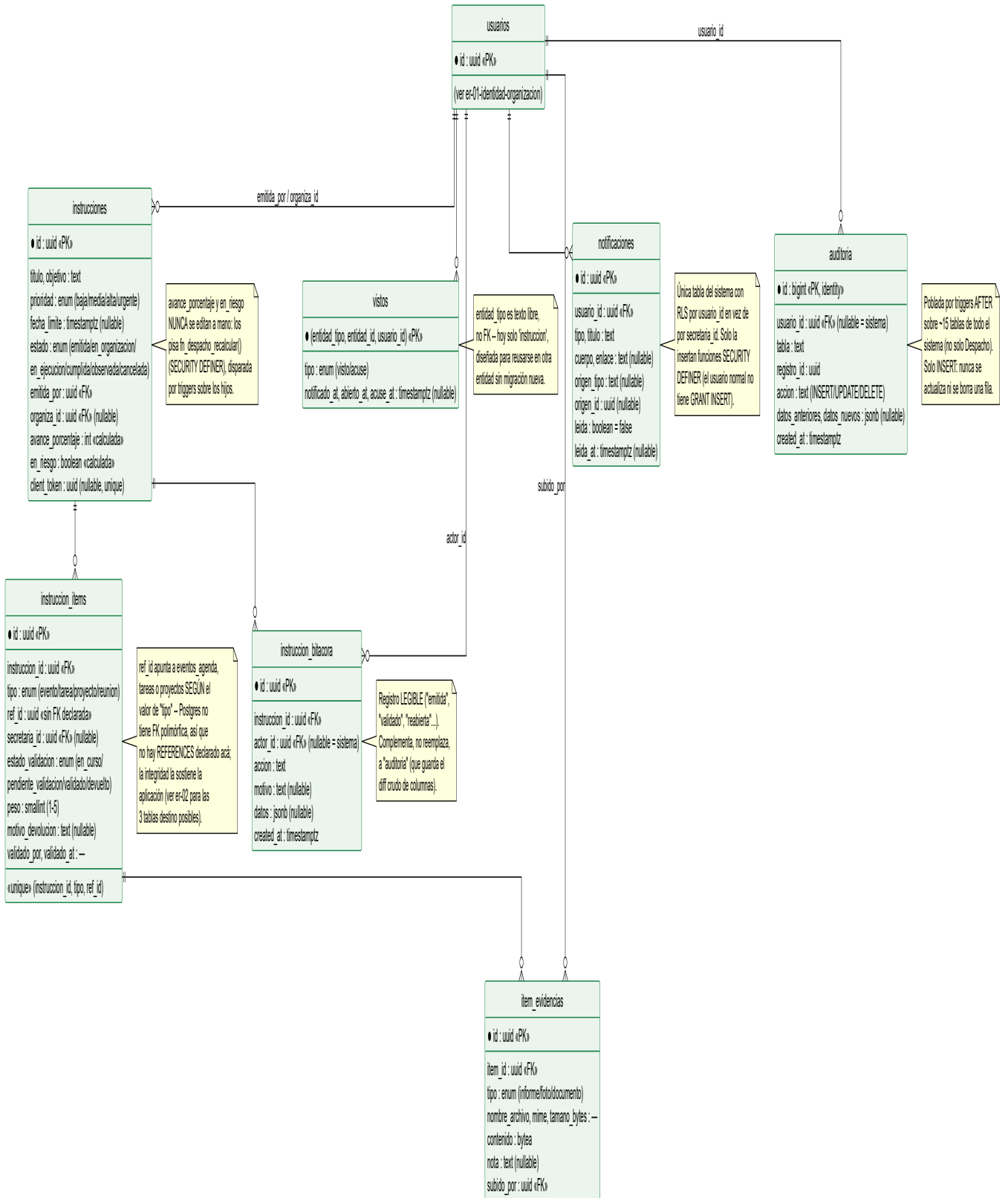


ER — Identidad y organización.

Entidad-Relación — Contenido institucional (trabajo diario de una secretaria)

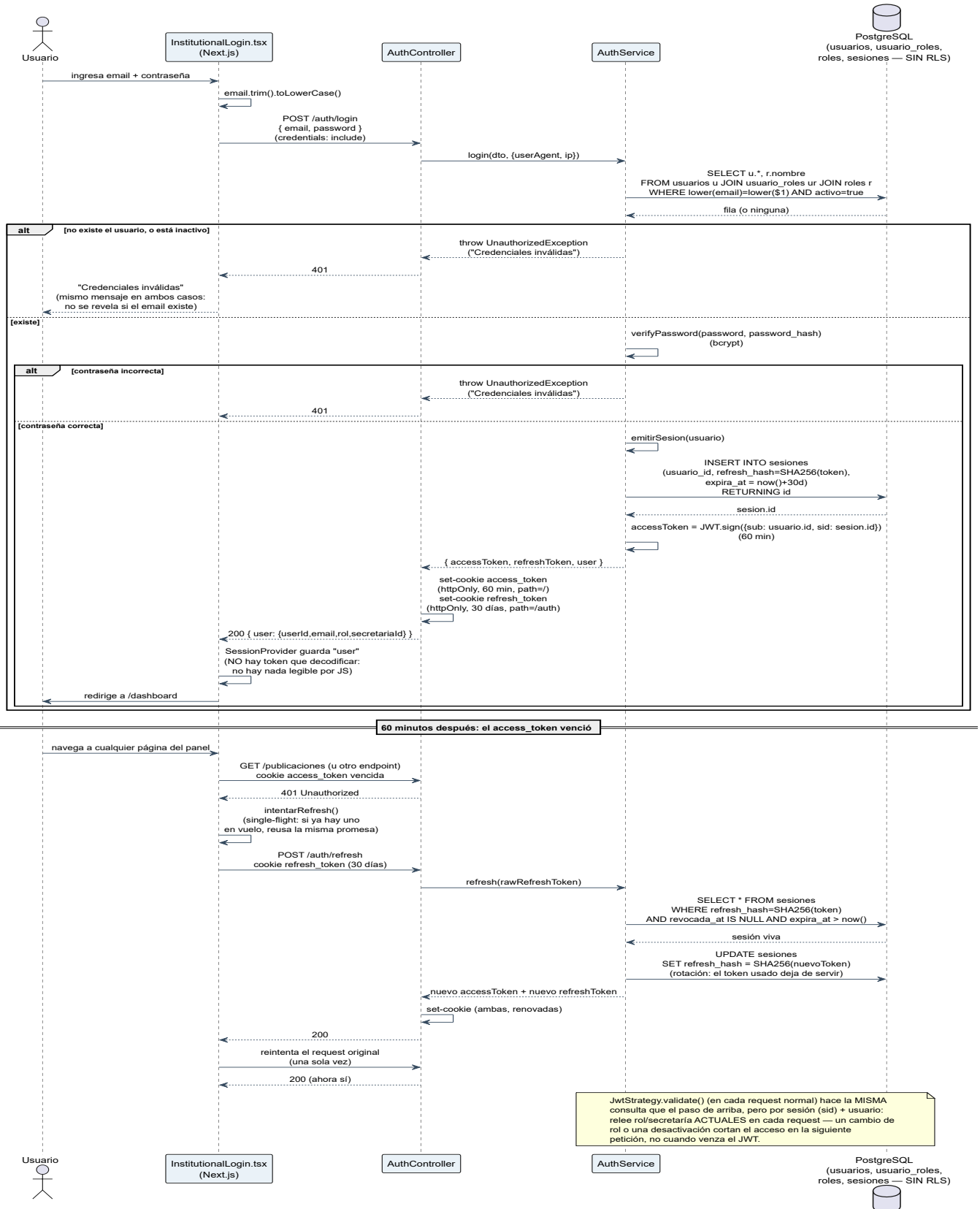


Entidad-Relación — Despacho (cadena de mando) y trazabilidad del sistema

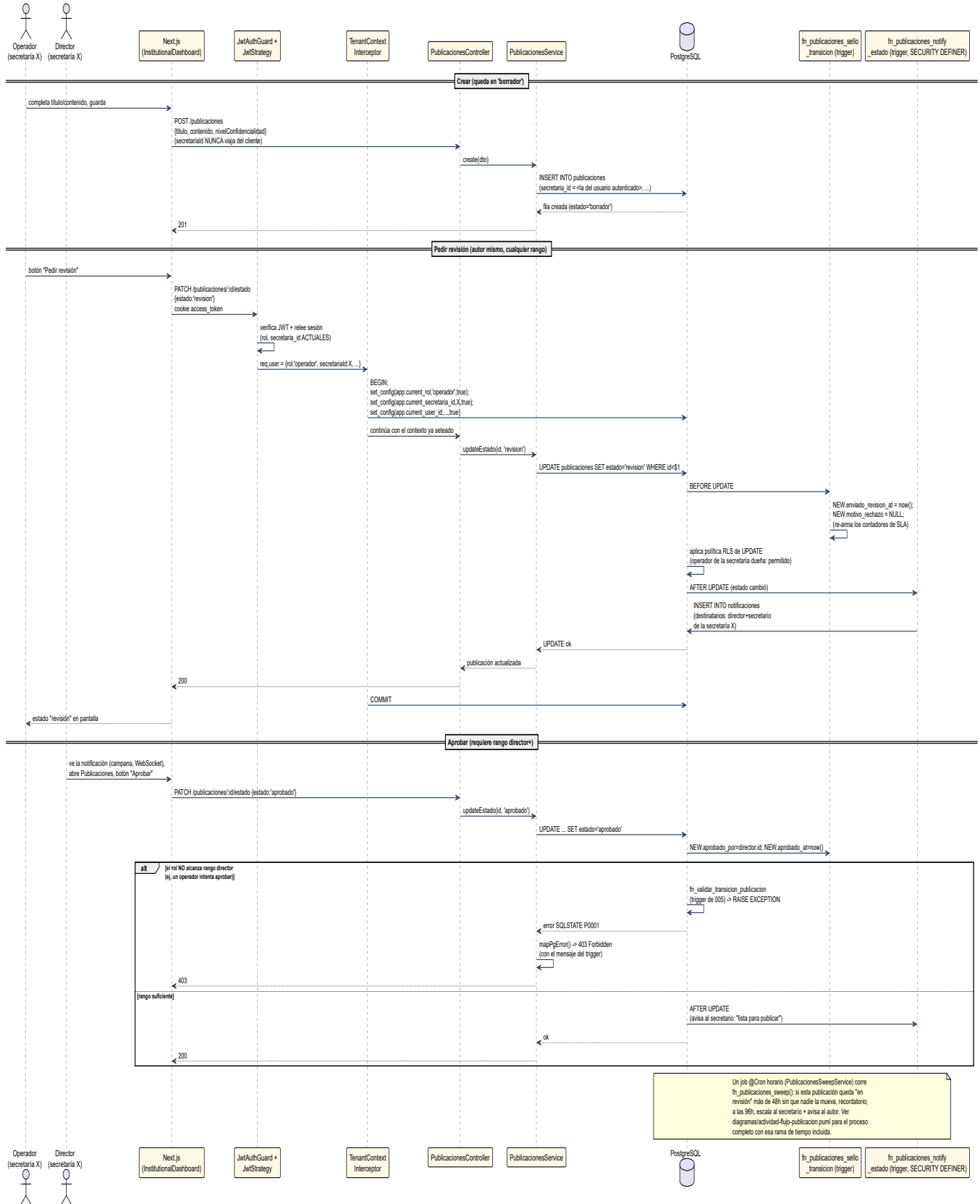


7.5 — Secuencia (6, un proceso técnico cada uno)

Secuencia — Login, emisión de sesión, y refresco automático de token

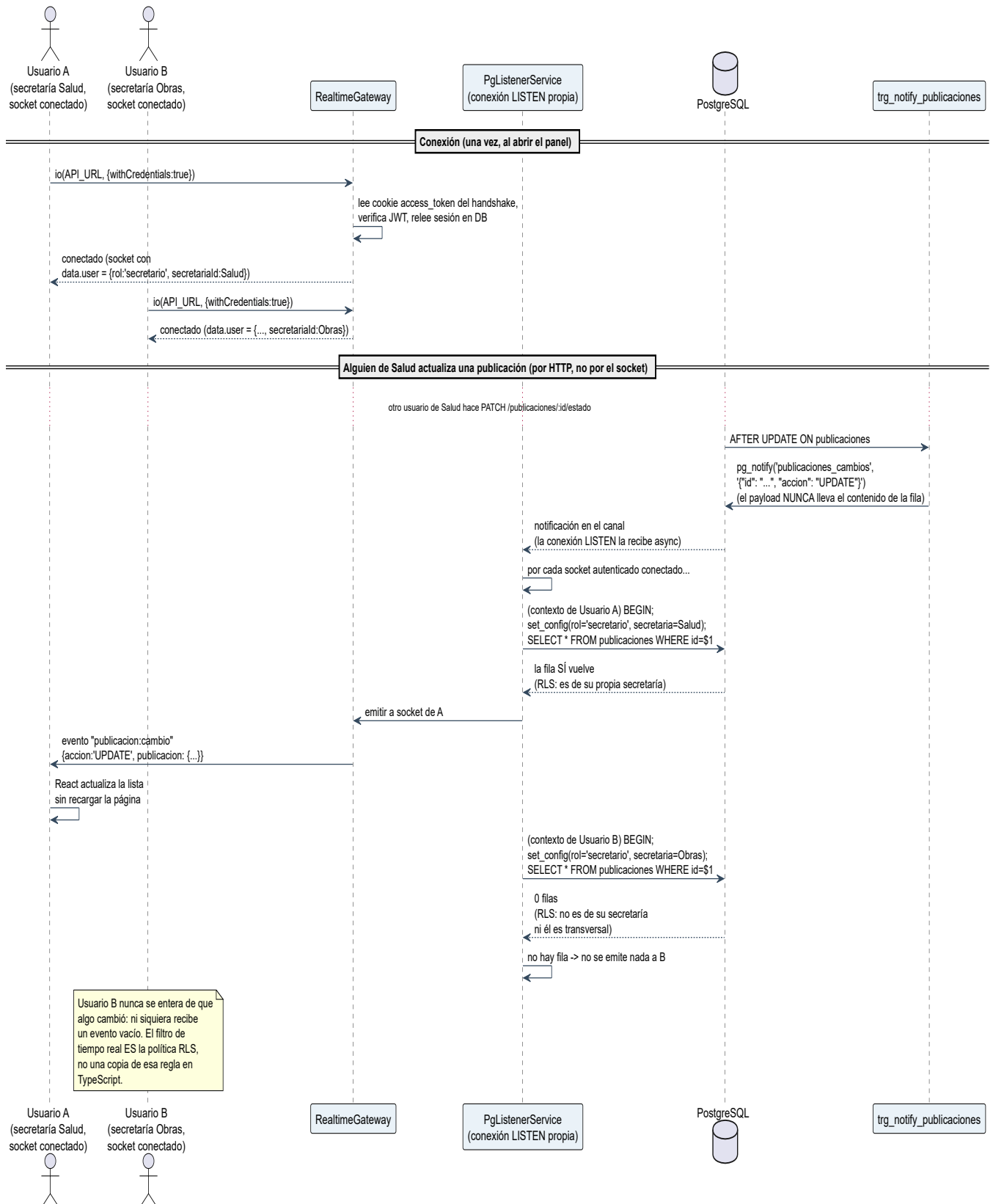


Secuencia — Publicación: crear y aprobar un comunicado (petición HTTP autenticada, capa por capa)



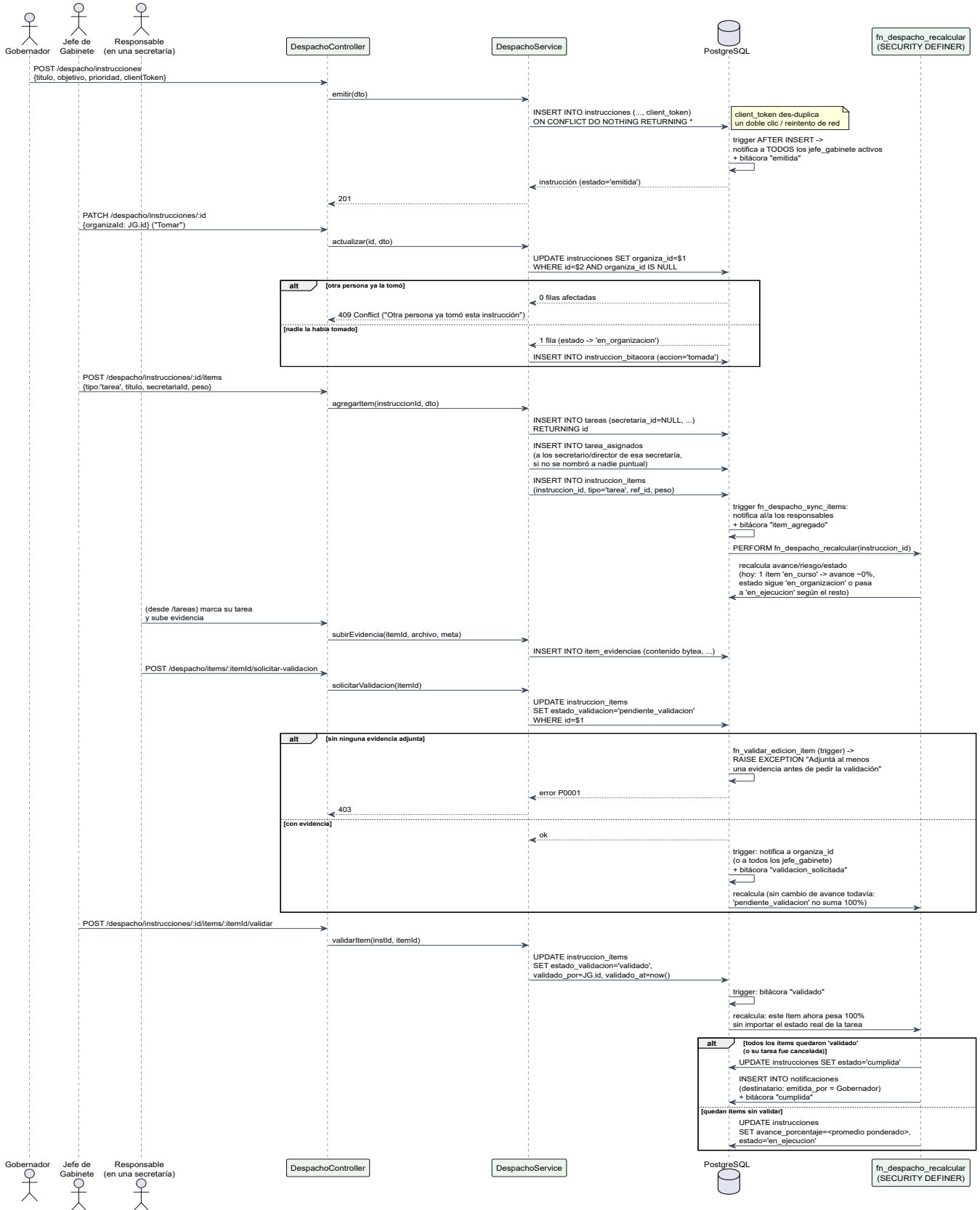
Secuencia — Crear publicación y flujo de aprobación.

Secuencia — Propagación en tiempo real (ejemplo: dos secretarías, un cambio de publicaciones)



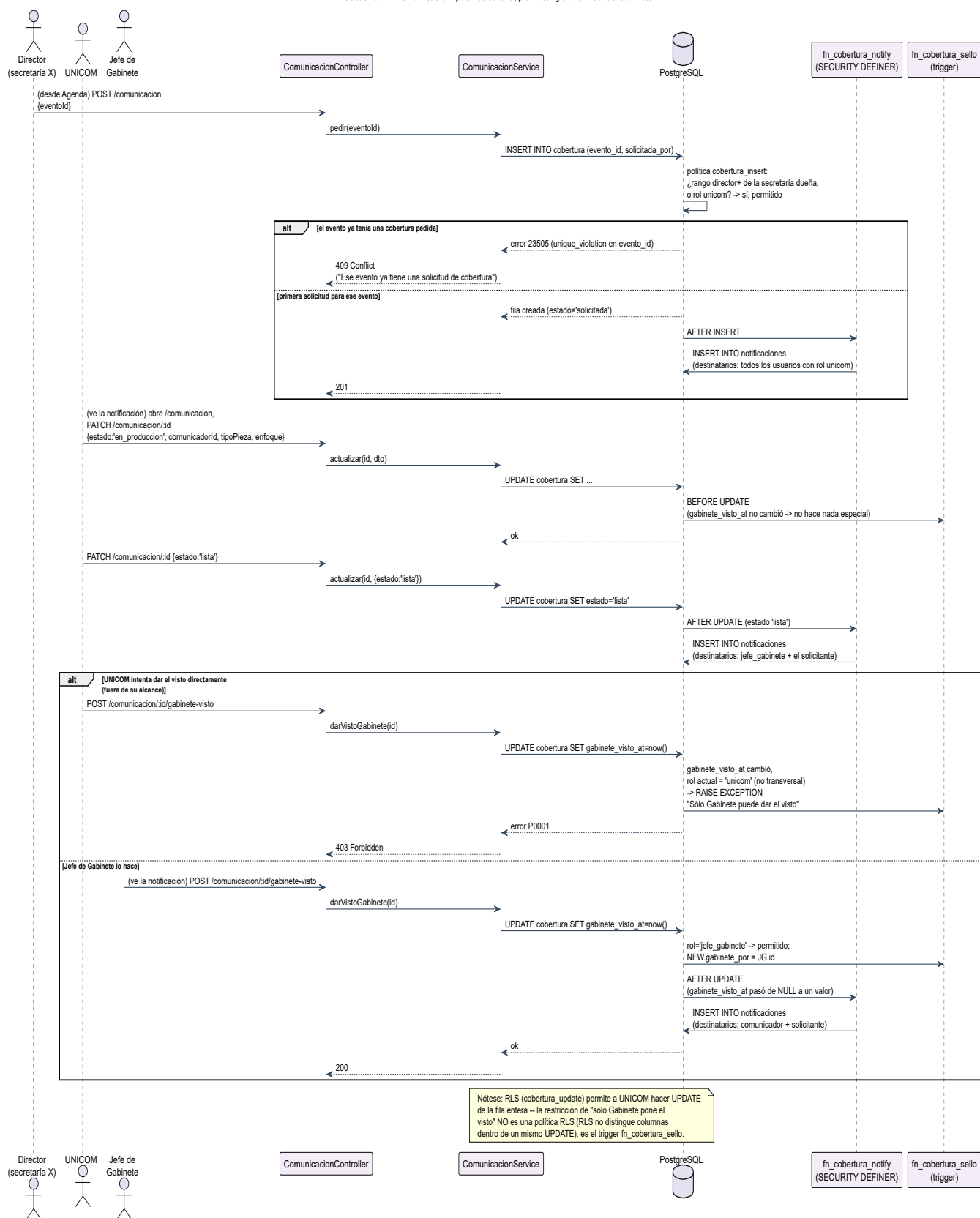
Secuencia — Actualización en tiempo real.

Secuencia — Despacho: emitir, desglosar, ejecutar y validar una instrucción



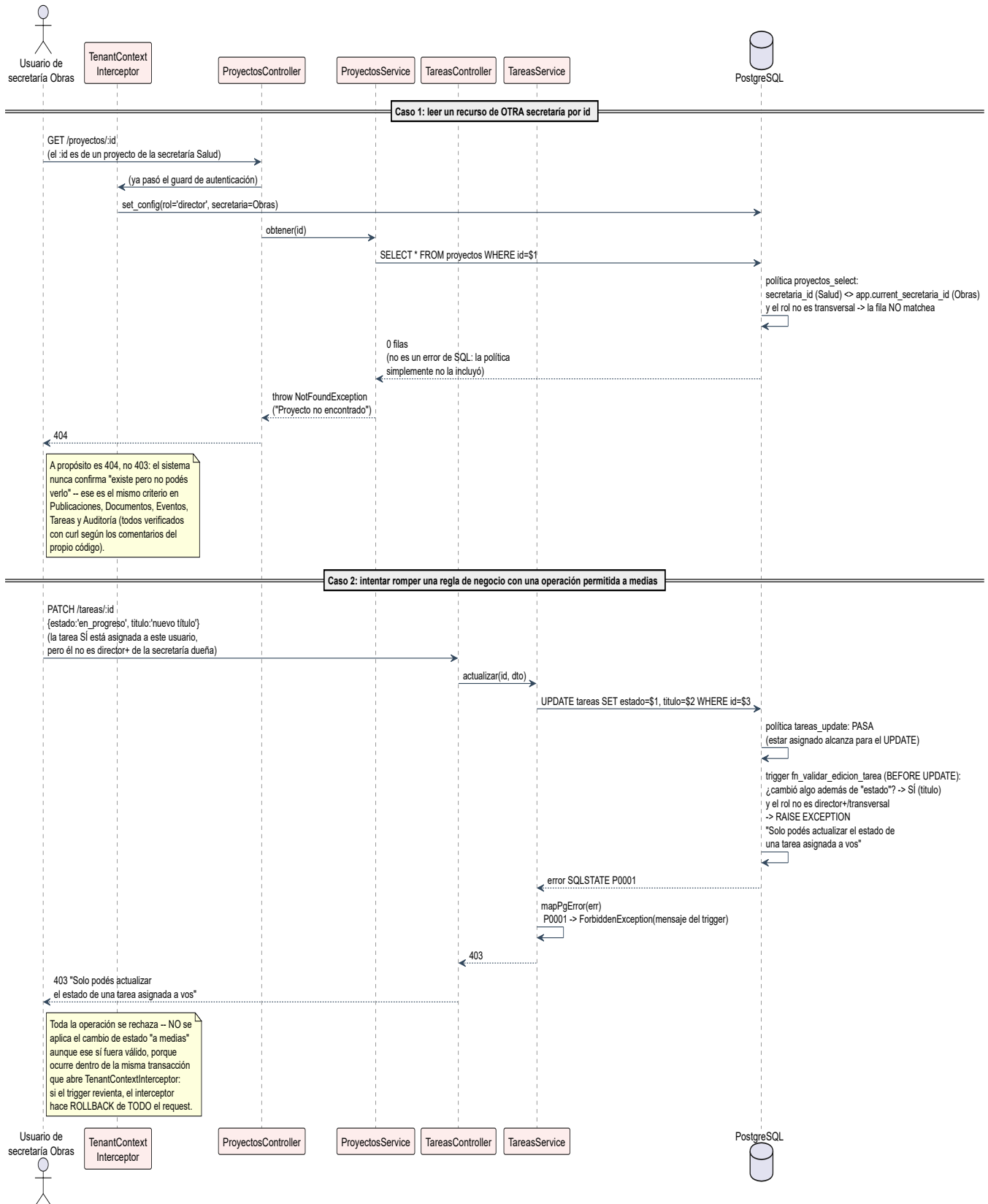
Secuencia — Despacho: emisión y validación.

Secuencia — Comunicación: pedir cobertura, planificar y dar el visto de Gabinete



Secuencia — Cobertura de comunicación.

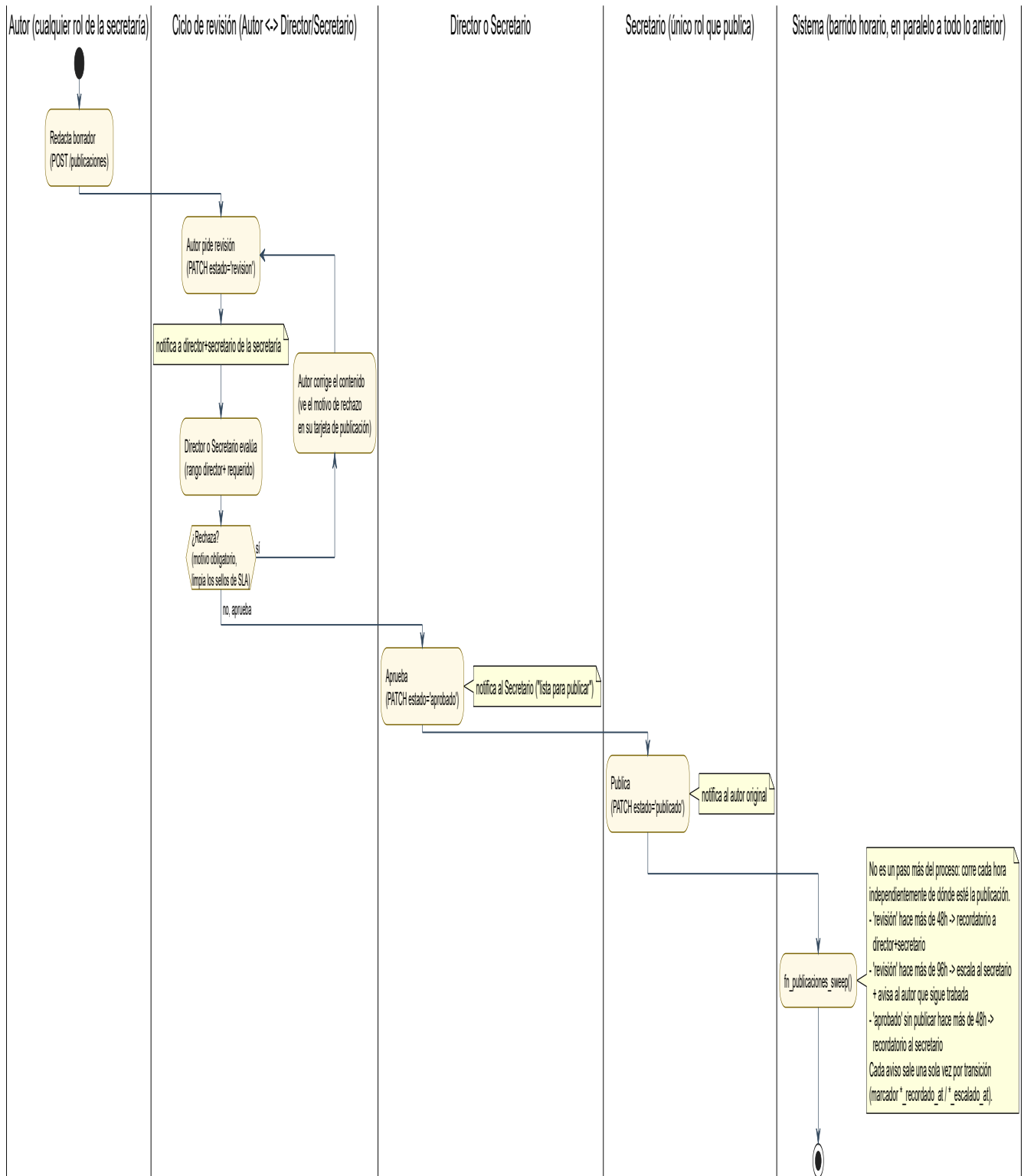
Secuencia — Dos formas de "no": denegado por defecto (404) vs. regla de negocio violada (403)



Secuencia — Los dos caminos de "no" (404 silencioso vs. 403 con motivo).

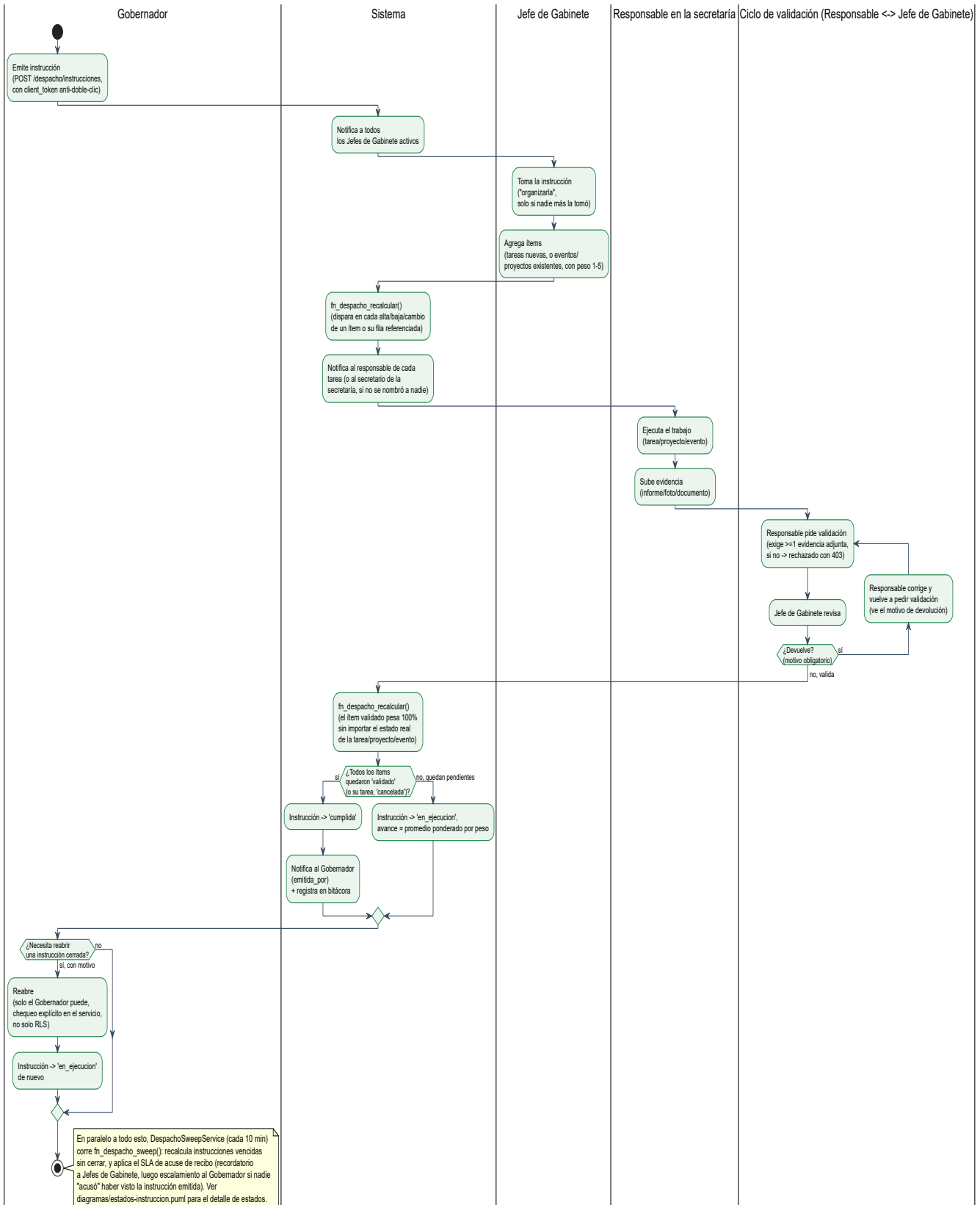
7.6 — Actividad (3, el proceso de negocio completo con sus ramas)

Actividad — Ciclo de vida de una Publicación, con rechazo y reintento



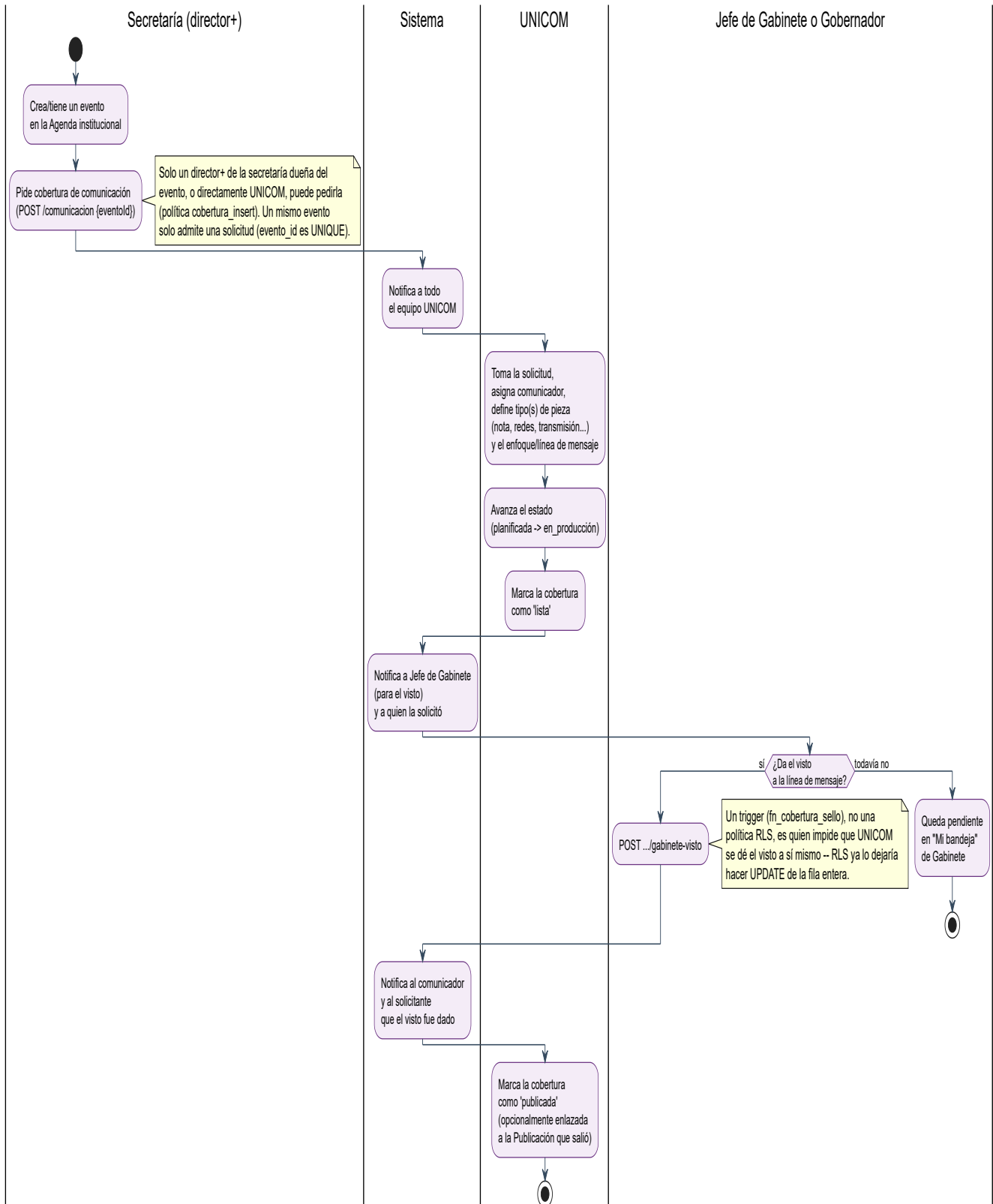
Actividad — Flujo completo de publicación, incluido el barrido horario en su propio carril.

Actividad — Ciclo de vida de una Instrucción de Despacho



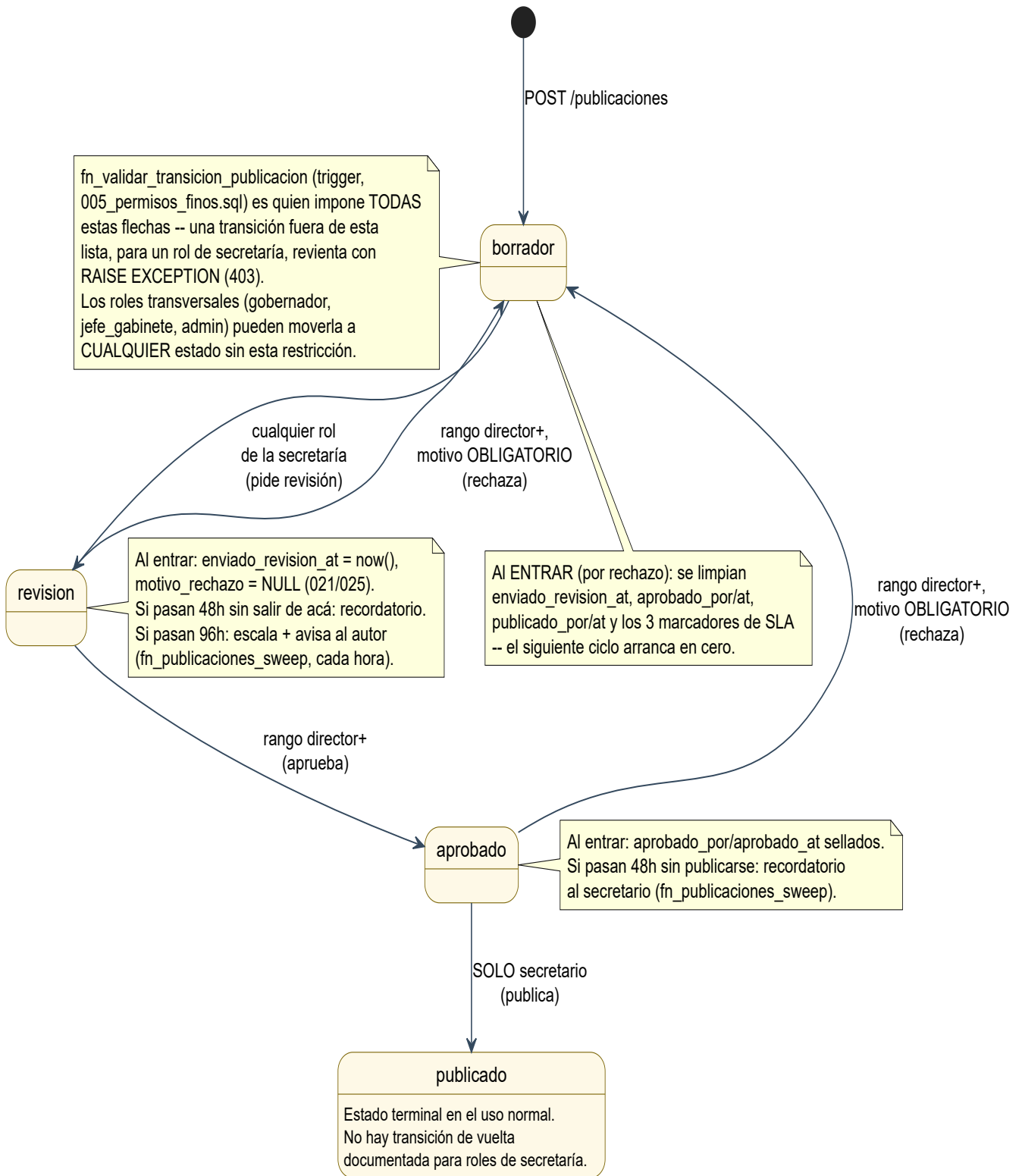
Actividad — Ciclo completo de una instrucción de Despacho.

Actividad — Solicitud y aprobación de cobertura de Comunicación (módulo v1)

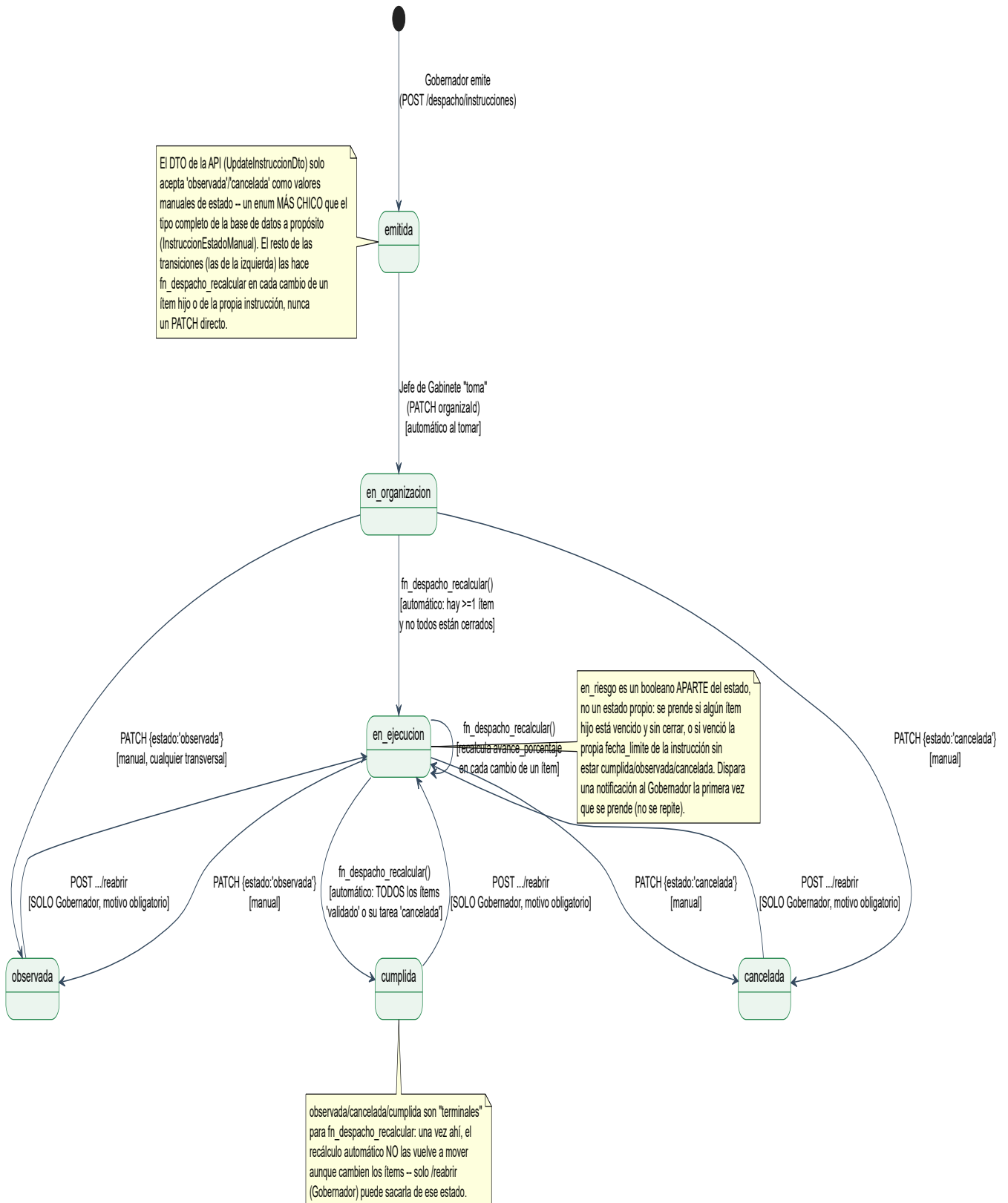


7.7 — Estados (5, el ciclo de vida de cada entidad con estado)

Estados — publicaciones.estado

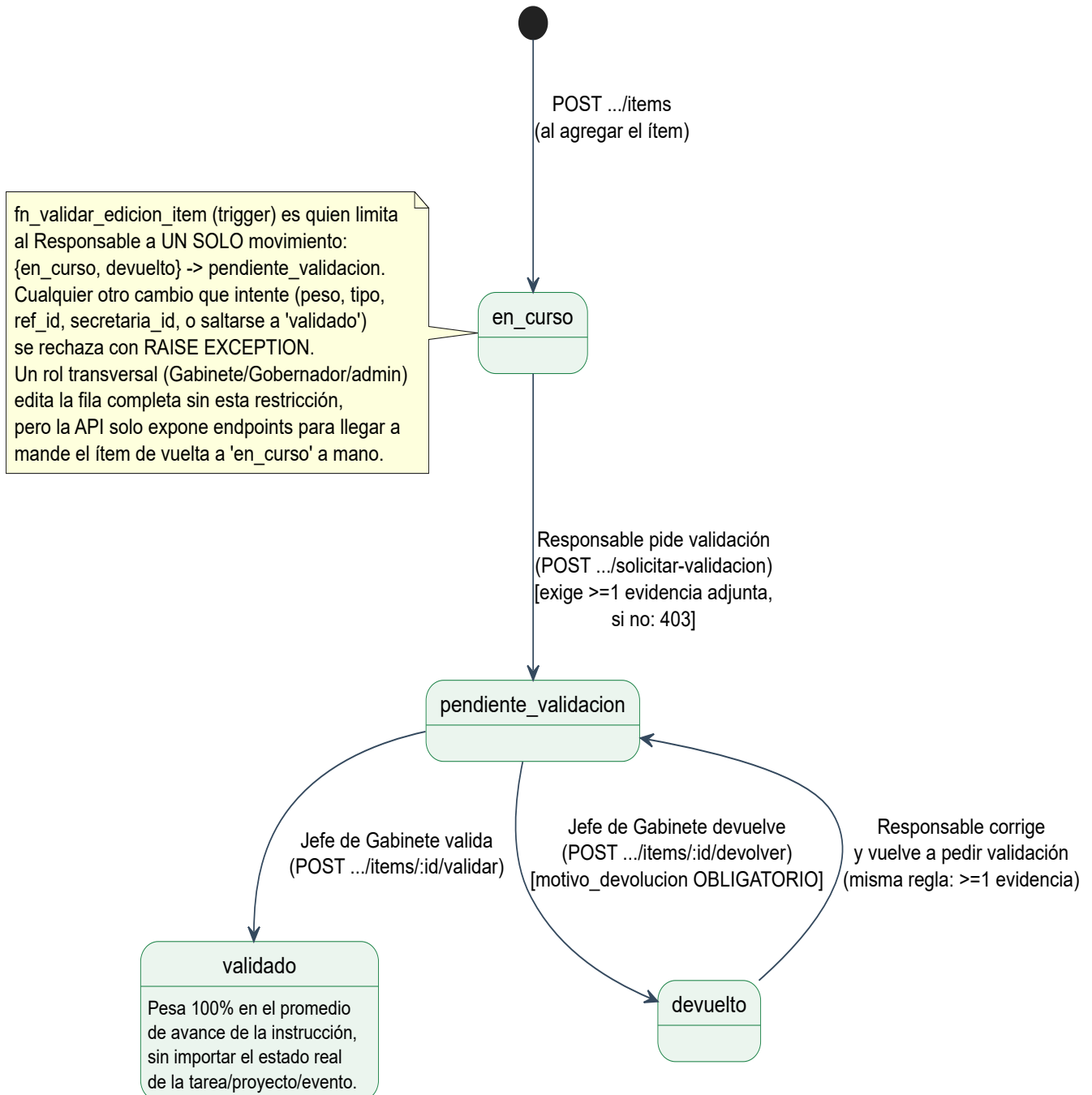


Estados — Publicación (todas las transiciones impuestas por trigger).

Estados — instrucciones.estado (Despacho)

Estados — Instrucción de Despacho.

Estados — instruccion_items.estado_validacion

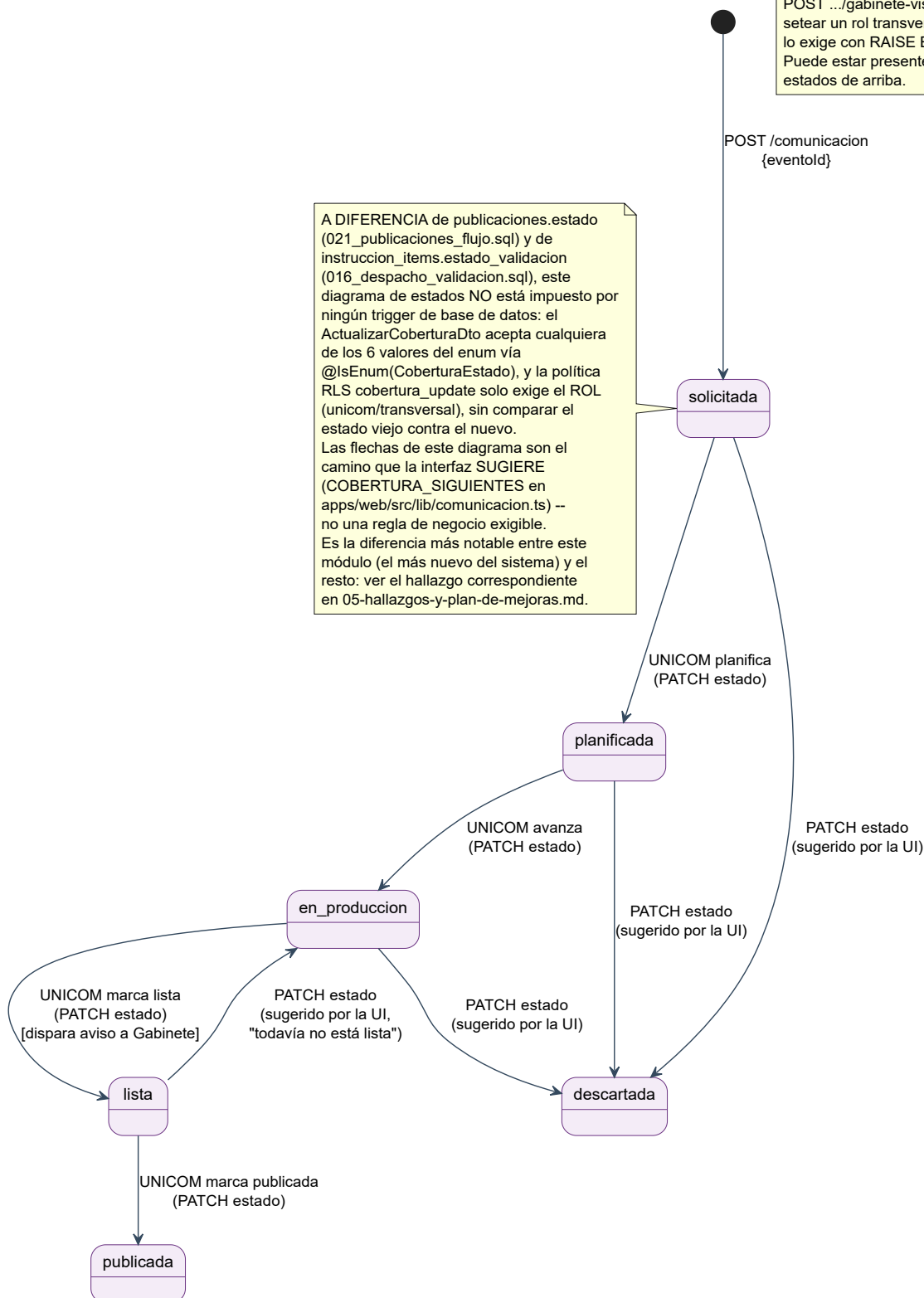


Estados — Ítem de instrucción.

Estados — cobertura.estado (Comunicación)

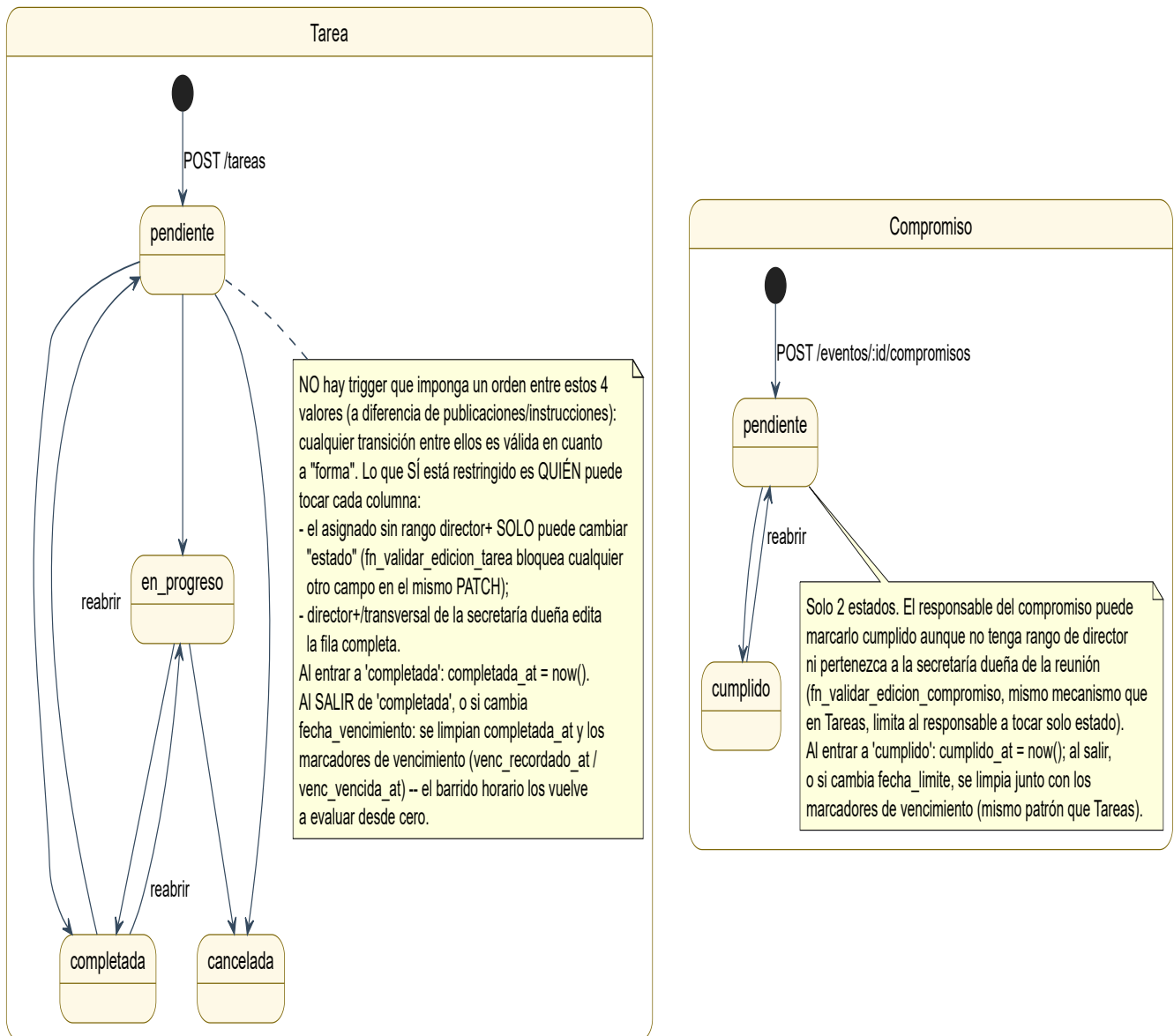
gabinete_visto_at es una COLUMNA aparte, no un estado más: se pone con POST .../gabinete-visto, solo la puede setear un rol transversal (fn_cobertura_sello lo exige con RAISE EXCEPTION si no). Puede estar presente en cualquiera de los estados de arriba.

A DIFERENCIA de publicaciones.estado (021_publicaciones_flujo.sql) y de instruccion_items.estado_validacion (016_despacho_validacion.sql), este diagrama de estados NO está impuesto por ningún trigger de base de datos: el ActualizarCoberturaDto acepta cualquiera de los 6 valores del enum vía @IsEnum(CoberturaEstado), y la política RLS cobertura_update solo exige el ROL (unicom/transversal), sin comparar el estado viejo contra el nuevo. Las flechas de este diagrama son el camino que la interfaz SUGIERE (COBERTURA_SIGUIENTES en apps/web/src/lib/comunicacion.ts) -- no una regla de negocio exigible. Es la diferencia más notable entre este módulo (el más nuevo del sistema) y el resto: ver el hallazgo correspondiente en 05-hallazgos-y-plan-de-mejoras.md.

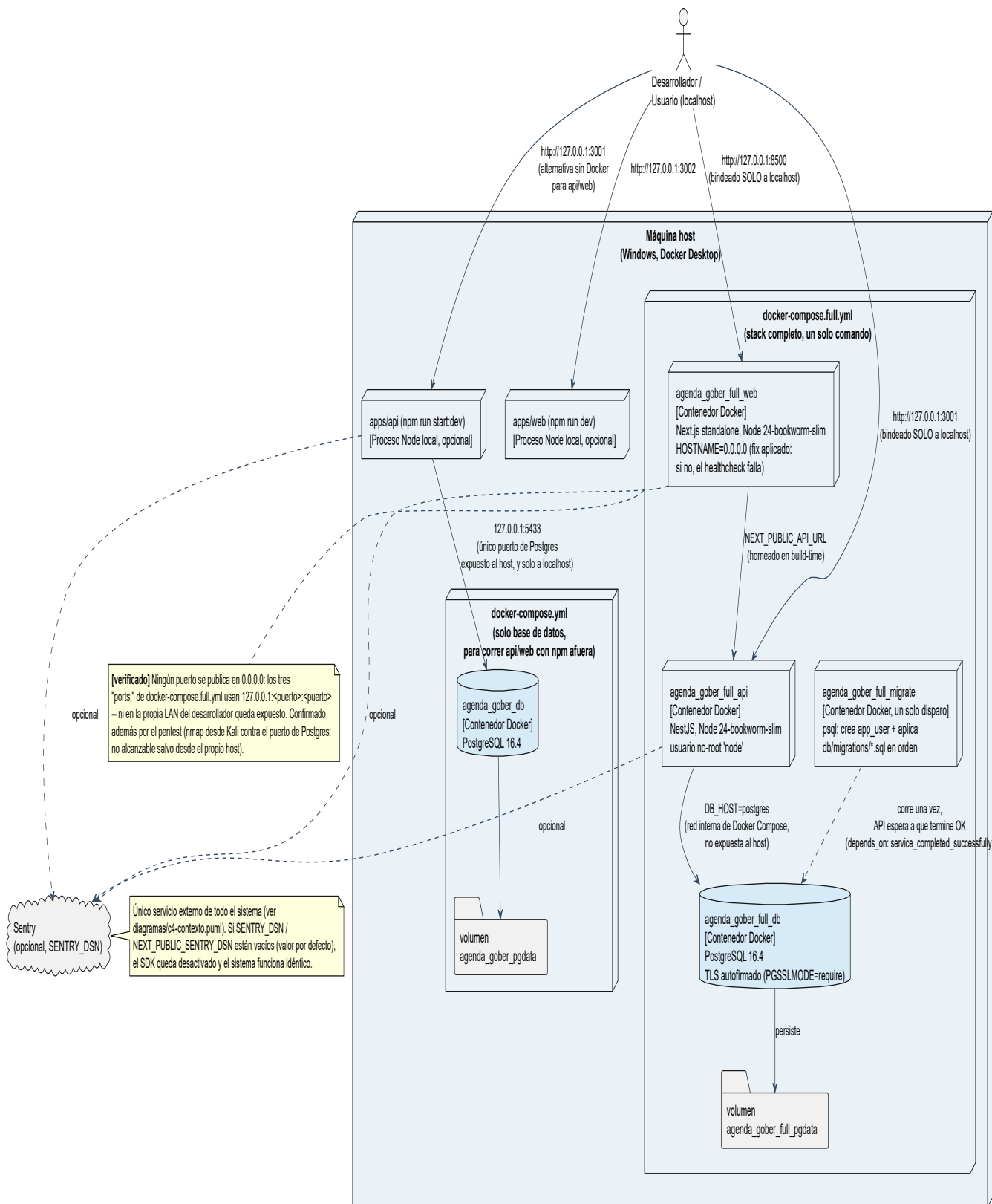


Estados — Cobertura (única sin trigger de transición — ver Hallazgo 7).

Estados — tareas.estado y compromisos.estado



Despliegue — topología real (Docker Compose local) + CI, sin CD a producción



Despliegue — todo vive hoy en la máquina de desarrollo; los puertos de **docker-compose.full.yml** se publican solo en **127.0.0.1**, confirmado con **nmap** desde otra máquina en el pentest.

Documento generado a partir de **docs/arquitectura/** (6 documentos narrativos + 25 diagramas validados) del repositorio AGENDA.GOBER, con una capa adicional de evaluación de dirección de arquitectura en la Sección 6. Fecha de corte: 12 de septiembre de 2026.